

Astana IT University

SADYKOV ABAY

**DevSecOps's view at construction of CI/CD pipelines.
Integration of Shift-left conception in modern methodology of
developing**

6B06301 — Cybersecurity

Diploma project

Supervisor
Makhambayeva Venera
Master of science in cybersecurity

Kazakhstan Republic
Astana, 2024

CONTENTS

Definitions	4
Designations and abbreviations	6
Introduction	7
1 THEORETICAL PART	9
1.1 Literature Review	9
1.2 About DevSecOps	9
1.2.1 Aims of DevSecOps	10
1.2.2 Importance of integrating security	12
1.2.3 Adapting AppSec to a DevOps	12
1.2.4 CAMS (Culture, Automation, Measurement, Sharing) framework to improve software quality	14
1.2.5 ADOC - conceptual model for automated DevSecOps	15
1.2.6 Comparative effectiveness and efficiency of DevOps and DevSecOps methodologies	16
1.3 Types of Security Testing	17
1.3.1 Static Application Security Testing (SAST)	17
1.3.2 Comparing Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST)	18
1.3.3 Robust Software composition analysis (SCA)	18
1.3.4 Interactive application security testing (IAST)	19
1.4 Monitoring	20
1.4.1 Site reliability engineering (SRE)	20
1.4.2 Improving visibility in application security	21
1.4.3 Spatio-temporal data analysis	22
2 PRACTICAL PART	24
2.1 Closed infrastructure	24
2.1.1 Plan of infrastructure	24
2.1.2 Using hypervisor VMWare ESXi for creating VMs	26
2.1.3 Autoconfiguration of VMs with Ansible	29
2.1.4 Set up of local Gitlab Repository	32
2.1.5 Set up of local Gitlab Runners with Docker and Shell executors	34
2.2 Back End API	35
2.3 Zabbix Monitoring	37
2.3.1 Install and configuration of Zabbix Server	37
2.3.2 Install and configuration of Zabbix Agent	38
2.3.3 Adding of Telegram Alerts	39
2.4 Gitlab DevSecOps CI/CD Pipeline	40
2.4.1 Secret Detection	41

2.4.2	Build	41
2.4.3	Static Application Security Testing (SAST)	42
2.4.4	Software Composition Analysis (SCA)	43
	Conclusion	44
	Bibliography	45
A	Ansible Configuration Files	47
1.1	default.conf	47
1.2	jail.local	47
1.3	filter.d	49
1.3.1	nginx-badbots.conf	49
1.4	my-install-script.yaml	49
1.5	debian.yaml	50
1.6	docker-debian-install.yaml	52
1.7	nginx-debian-install.yaml	53
B	FastAPI Back End	55
2.1	app/	55
2.1.1	configs/	55
2.1.2	database/	55
2.1.3	exceptions/	57
2.1.4	routers/	57
2.1.5	schemas/	58
2.1.6	responses/user_schema.py	59
2.1.7	services/	59
2.1.8	main.py	61
2.1.9	utils.py	62
2.2	.env	62
2.3	requirements.txt	62
2.4	Dockerfile	63
2.5	docker-compose.yml	63
C	Gitlab CI/CD	66
3.1	.gitlab-ci.yml	66

DEFINITIONS

Following terms are used in this work:

API	A set of rules and specifications that define how software components interact with each other. APIs enable integration and data exchange between systems.
REST	An architectural style for building web services, often used for APIs. REST emphasizes statelessness, scalability, and a uniform interface.
WAF	A security layer that filters and monitors HTTP traffic at the application level, protecting web applications from common attacks like SQL injection and cross-site scripting (XSS).
UFW	A widely used frontend for managing iptables (Linux's built-in firewall tool). Its goal is to provide a user-friendly interface for common firewall tasks.
SSL	A security protocol used to establish encrypted links between a web server and a browser.
DevOps	a cultural and process approach that emphasizes collaboration, automation, and rapid feedback between development and operations teams. Its aim is to optimize the delivery process and improve software quality.
SecOps	The application of security principles and practices within an IT operations environment. Focuses on real-time monitoring, incident response, and threat protection.
QA	A systematic process of ensuring that software products meet defined quality standards throughout development. Involves activities like testing and reviews.
SRE	A discipline that applies software engineering principles to IT operations, balancing reliability with efficient feature delivery. Focuses on automation, observability, and scalable systems.
DevSecOps	An evolution of DevOps that emphasizes integrating security practices and tools from the beginning of the software development lifecycle (SDLC). Aims to create a culture of shared security responsibility.
AppSec	A field concerned with securing software applications at various stages of development and deployment.

DAST	A black-box testing approach that examines a running application to find vulnerabilities exploited at runtime.
SAST	A white-box testing method that analyzes source code without executing it, focusing on identifying potential security flaws in the code itself.
IAST	A hybrid approach that combines aspects of SAST and DAST, often using runtime tooling for deeper vulnerability analysis.
SCA	The process of examining third-party components and open source dependencies in a software application to identify known vulnerabilities and license risks.
CSA	An approach of integrating various security testing and monitoring tools throughout the SDLC to make earlier security changes and provide continuous feedback.
CI	A practice where developers often merge code changes into a common repository, running automated builds and tests.
CD	An extension of CI that automates the deployment of software to test or production environments, focusing on fast and reliable releases.
OWASP	A non-profit foundation dedicated to improving web application security. They provide resources, tools, and standards, including their famed "OWASP Top 10" list of common web vulnerabilities.

DESIGNATIONS AND ABBREVIATIONS

Following designations and abbreviations are used in this work:

API	Application Programming Interface
REST	REpresentational State Transfer
WAF	Web Application Firewall
UFW	Uncomplicated Firewall
SSL	Secure Sockets Layer
DevOps	Development Operations
SecOps	Security Operations
QA	Quality Assurance
SRE	Site Reliability Engineering
DevSecOps	Development Security Operations
AppSec	Application Security
DAST	Dynamic Application Security Testing
SAST	Static Application Security Testing
IAST	Interactive Application Security Testing
SCA	Software Composition Analysis
CSA	Continuous Security Analysis
CI	Continuous Integration
CD	Continuous Delivery
OWASP	Open Worldwide Application Security Project

INTRODUCTION

Work relevance. The topic has recently become popular in IT companies because of the increased number of attacks on major IT companies in recent years. It causes damages to the campaigns and also forces cybersecurity and automation professionals to rethink their thinking and methods in building of infrastructure. DevSecOps culture is an innovative response to that problem since it introduces a new vision that in turn tries to bring together system administration, automation, product development, and cybersecurity professionals into a cross-platform team [1, 2].

Goal of the work. The aim of the work is to display how integration of Shift-left conception could increase security and time-efficiency of AppSec's team

Research object. The main object of study in this paper is DevSecOps methodology, particularly at what points the differences between typical DevOps ways of thinking in building conventional infrastructure stand out. As subjects of the work, the different types of security analysis of web applications, the cases in which they are applied, and the tools that will be used for the analysis itself will be discussed.

Novelty. The novelty of the work is that it illustrates in concrete steps how to integrate different types of security analysis like SAST, DAST, IAST, SCA, CSA in practice by creating special stages in a standard Gitlab CI/CD pipeline. In addition, it shows which tools are used for each type of analysis, which are better and which are worse in certain situations

Research methodology. The research methodology explores the integration of various security analysis techniques within the DevSecOps methodology using a qualitative research approach. This method, chosen to provide in-depth understanding, includes a comprehensive review of web application analysis types, practical implementation of DevSecOps principles in the GitLab CI/CD pipeline, establishing a secure network connection, organizing quality and stability control using monitoring tools, and developing and deploying a server-side test web application based on the RESTful API paradigm. For the literature review were used sources from: official documentations of tool vendors, various web courses on application security from IBM, YandexCloud, scientific articles from conferences, and some articles on the Habr platform provided by Yandex.

Practical relevance of the work. The practical relevance of the work is to change DevOps's thinking of CI/CD pipeline constructions to more effective and secure way.

Objectives:

- 1 Explore the history of creating DevSecOps
- 2 Explain how to set up closed infrastructure with cloud native services for secure developing

- 3 Automate the uplift of servers with configured firewall, dependencies and policies for them
- 4 Develop and deploy a test server-side web application based on the RESTful API paradigm
- 5 Organize control of server state by using monitoring tools and additional management dashboards
- 6 Implement a CI/CD pipeline using the core concepts of DevSecOps

1 THEORETICAL PART

1.1 Literature Review

A systematic literature review by Sanchez-Gordon and Colomo-Palacios emphasizes the key role of cultural transformation for successful DevSecOps adoption. While technical tools and processes are important, the study highlights collaboration, knowledge sharing, continuous feedback, shared accountability, and trust as fundamental cultural elements. DevSecOps requires breaking down silos, integrating security into the early stages of development, and increasing security awareness among all team members. These cultural changes promote proactive security that aligns with the business imperative to prevent costly breaches, the ability to combine speed and security, and the increasing market and regulatory demands for internally secure software. The findings indicate the potential for DevSecOps to become mainstream, which calls for further empirical research to explore the practical aspects and outcomes of this cultural shift in organizations. [3]

1.2 About DevSecOps

During a joint session hosted by MITRE and ATARC, it was highlighted that government organizations and industry face significant challenges when it comes to securing cloud environments and maintaining robust defensive cyber operations (DCO). The primary challenge is the perceived loss of control and visibility once workloads move into the cloud provider's domain. This raises questions about quantifying the risks in the CSP infrastructure and the effectiveness of available defenses. Adapting to cloud computing requires cybersecurity teams to rethink their roles and responsibilities, particularly in managing relationships with external stakeholders such as cloud service providers (CSPs).

Additionally, the shared responsibility model in cloud environments presents challenges for cyber operations teams in delineating ownership of security controls, integrating systems seamlessly, and managing the vast amount of data generated by CSPs' numerous cybersecurity tools. Security in the cloud requires new skill sets, with an increasing focus on automation, data science, and risk management. Addressing these challenges is greatly aided by specialized research in areas such as modeling cloud-specific threats, optimizing multi-cloud incident response strategies, and assessing risk. [4]

DevSecOps represents a significant change in software development culture, advocating for security to be an integral component from the beginning of a project, rather than a late, potentially disruptive step. François Reynaud emphasizes the importance of collaboration and knowledge transfer between different disciplines to break down the traditional 'silos' that separate security teams from developers. This approach aims to establish shared responsibility

for secure development practices and reduce friction between development and security teams.

DevSecOps emphasizes 'security shift left,' integrating security considerations into non-functional requirements and prioritizing threat modeling early in development. This approach, combined with automated security testing in the CI/CD pipeline, significantly reduces the risks and costs associated with fixing vulnerabilities later in the software development life cycle (SDLC). Additionally, the DevSecOps model promotes 'security champions' - developers with advanced security training who share knowledge and cultivate security awareness within their teams.

These fundamental tenets of DevSecOps have the potential to change the software development process with far-reaching benefits. Organizations that use this methodology can expect an overall reduction in risk through proactively developed, inherently secure applications. Efficiency gains and process optimization stem from implementing security early in the SDLC, eliminating the need for costly intervention at later stages. DevSecOps fosters a culture of collaboration, emphasizing that quality secure products are a shared responsibility between developers and security personnel. It is important to note that this statement is subjective and has been left unchanged. [5]

The study highlights the significance of integrating security practices during the initial stages of software development rather than considering them as an afterthought. Automation plays a crucial role in DevSecOps by ensuring a consistent, scalable, and repeatable approach to security. Continuous testing, along with rapid feedback loops, enables development teams to promptly identify and rectify potential security vulnerabilities, resulting in a safer development process.

To successfully implement DevSecOps, organizations must undergo a cultural shift. This involves fostering collaboration between development, operations, and security teams to break down silos and establish a shared understanding of security responsibilities. Additionally, adapting existing IT governance models to prioritize security and agility is crucial to meet the unique requirements of DevSecOps approaches, particularly in complex system environments. [6]

1.2.1 Aims of DevSecOps

DevSecOps aims to overcome the limitations of traditional security practices by integrating security principles directly into the software development lifecycle (SDLC). While its importance is increasingly recognized, many organizations face significant hurdles in establishing effective and impactful DevSecOps practices. A major challenge stems from a misunderstanding of how security activities truly mesh with the fast-paced, iterative nature of DevOps methodologies. Organizations may assume that simply purchasing and automating security tools

guarantees robust protection, leading to ineffective deployment and ultimately frustration from developers.

To ensure meaningful DevSecOps success, practitioners must prioritize shifting security practices left. Static analysis tools should be deployed as early as the developer's IDE within the SDLC – empowering teams to find and fix vulnerabilities proactively. Additionally, 'breaking the build' when critical issues are detected forces timely resolution in line with DevSecOps principles. However, tools alone can't address the human factor: open lines of communication between development, operations, and security teams are paramount. Collaborative processes must support knowledge exchange and rapid response to evolving risks.

While organizations recognize the significance of DevSecOps, implementation demands significant cultural and mindset adjustments. Adopting a risk-based approach is vital, tailoring security controls to match specific business needs and risk appetites. Education and cross-training foster a 'security-as-shared-responsibility' mindset, promoting greater cooperation. Effective DevSecOps implementation starts with small successes: one team can adopt and perfect their pipeline, establishing a powerful and replicable blueprint for scaling security efforts across the organization. [7]

DevSecOps seeks to seamlessly integrate security practices throughout the software development lifecycle (SDLC). However, many organizations struggle to effectively adopt this approach due to misconceptions and ineffective strategies. Tanya Janca highlights the common misunderstanding that DevSecOps success relies solely on tools. To be effective, tools require customization and integration alongside open communication channels between development, operations, and security teams. This collaboration facilitates knowledge sharing and proactive problem-solving throughout the SDLC.

Successful DevSecOps requires cultural change; it's not simply about acquiring technology. Integrating security early into the development process with practices like static analysis encourages finding vulnerabilities during initial coding. 'Breaking the build' processes, triggered by critical issues, enforce timely fixes and prevent cascading security problems. Most importantly, DevSecOps demands that security responsibilities be understood and shared across all teams. Fostering a culture of continuous communication and collaborative security is vital.

Janca emphasizes that effective adoption often starts small – focusing on success within individual teams before scaling across the organization. Future DevSecOps research needs to optimize adaptation across complex environments and define concrete metrics to illustrate the impact and return on investment in security-focused development. This can foster continued support and improvement for DevSecOps success. [8]

1.2.2 Importance of integrating security

This position study highlights the importance of integrating security practices directly into the software development lifecycle. A key strategy is mapping observable attacker tactics (using frameworks like Mitre’s ATT and CK) to specific source code weaknesses like Common Weakness Enumerations (CWEs). This tight link between malicious actors and vulnerable code is vital for developers to prioritize remediation efforts. The focus isn’t simply on maintaining clean code but rather understanding which vulnerabilities pose the greatest security risk to the business.

By combining observations from operations staff (Ops) with the code vulnerability knowledge held by developers (Dev), the DevSecOps approach aims to minimize "technical debt" associated with security weaknesses. This 'debt' accrues interest over time - unpatched vulnerabilities increase in potential damage. This study argues that tight feedback loops facilitated by DevSecOps help address high-priority weaknesses quickly, limiting technical debt exposure. It proposes a new security-driven metric for technical debt measurement, emphasizing a software weakness’s potential business damage rather than purely technical complexity. Further research is advocated into understanding the entire lifecycle of these weaknesses, from the attack exploit to vulnerability patching, in real-world enterprise environments. [9]

1.2.3 Adapting AppSec to a DevOps

The literature review synthesizes insights from Rebecca Deck’s white paper "Adapting AppSec to a DevOps World" along with supporting literature, providing a comprehensive overview of the challenges and strategies for integrating application security within DevOps practices

The advent of DevOps has significantly transformed software development, focusing on continuous integration, continuous delivery, and the cohesive operation of development and operations teams. This shift poses considerable challenges for traditional Application Security (AppSec) practices, which historically relied on extensive manual testing and lengthy security assessments. This literature review delves into how AppSec can be integrated into a DevOps framework, emphasizing the importance of automation, the creation of abuse cases, and the evolving roles of security teams.

In traditional software development, security is often enforced through rigorous checks at various stages of the development process. These checks, involving multi-day static and dynamic analysis performed by a few security experts, are incompatible with the fast-paced DevOps environment where multiple software releases can occur daily. Research by Kim et al. (2016) indicates that these traditional methods can create significant bottlenecks, highlighting the need for more integrated and automated security processes.

DevOps prioritizes automation to ensure rapid and safe software delivery, minimizing the need for extensive manual interventions. By embedding security testing within existing DevOps toolchains, organizations can automate security processes without disrupting development workflows. Automated unit and integration tests are essential for maintaining security standards while keeping up with the rapid deployment cycles typical of DevOps environments. Tools like the Mocha testing framework help automate abuse case testing, making it an integral part of the CI/CD pipeline

A key aspect of integrating AppSec into DevOps is the development of abuse cases. Unlike traditional use cases that describe expected software behavior, abuse cases detail how software should behave under malicious conditions. Deck (2021) emphasizes that incorporating abuse cases into the SDLC ensures that security is built into the software from the beginning. Threat modeling tools such as the Microsoft Threat Modeling Tool and OWASP Threat Dragon assist in creating comprehensive abuse cases by allowing teams to visualize potential threats and vulnerabilities

In the DevOps context, the role of security teams shifts from gatekeepers to enablers, helping developers embed security into their workflows and automating routine security tasks. This transition requires close collaboration between security teams and developers, ensuring that security becomes a fundamental part of the development process rather than an afterthought. By fostering a culture of continuous improvement and utilizing automated tools, security teams can focus on strategic tasks such as developing new abuse cases and updating threat models based on emerging threats.

Continuous improvement is a core principle of DevOps, and it extends to AppSec practices. Monitoring the effectiveness of abuse cases and updating threat models in response to new vulnerabilities ensures that security measures evolve alongside the software. This proactive approach helps identify and mitigate risks early in the development cycle, reducing the likelihood of security breaches after deployment.

Integrating AppSec into a DevOps environment requires a fundamental shift from traditional manual security practices to automated, developer-centric approaches. By embracing automation, developing robust abuse cases, and redefining the roles of security teams, organizations can achieve the goals of rapid software delivery and strong security. This integration enhances security posture while aligning with the core DevOps principles of continuous improvement and efficiency. [10]

1.2.4 CAMS (Culture, Automation, Measurement, Sharing) framework to improve software quality

In the 2017 International Conference on Advances in ICT for Emerging Regions (ICTer), Pulasthi, Perera, Roshali, Silva, and Indika presented a comprehensive study titled “Improve software quality through practicing DevOps”. The objective of their study was to understand how DevOps practices can improve software quality in organizations, specifically focusing on the role of CAMS (Culture, Automation, Measurement, Sharing) framework in this improvement.

The authors positioned their study within a broader discussion of software development methodologies, specifically contrasting DevOps with traditional approaches such as the waterfall model and Agile practices. The literature review highlights the evolution of software development methods, noting that while Agile methodologies address some of the limitations of traditional models by promoting agility and responsiveness to change, DevOps extends these principles by integrating operations with development to enhance continuous integration and delivery.

The methodology used by Perera et al. was based on both quantitative and qualitative analysis. They formulated hypotheses about the impact of DevOps practices on software quality, which were tested using Pearson correlation coefficients derived from survey data. The study also included linear regression analysis to examine the relationship between DevOps practices and various quality dimensions defined in ISO 9126, including functionality, reliability, and maintainability.

The results showed a significant positive correlation between DevOps practices and improved software quality, which supports the hypothesis that DevOps implemented using CAMS framework leads to improved software quality. Notably, automation was found to be a critical factor, suggesting that the more automation in the DevOps cycle, the higher the software quality. This finding is consistent with current industry trends emphasizing the need to automate testing and deployment processes to improve efficiency and reliability.

This study contributes to the academic understanding of how integrating development and operations, as prescribed by DevOps, can specifically improve software quality. Using the CAMS framework as a theoretical lens, the study provides a structured approach to analyze the impact of cultural shifts, automation, measurement techniques, and sharing within development teams on software outcomes. The study not only supports the theoretical underpinnings of DevOps, but also offers empirical evidence linking these practices to improved software development quality.

The work of Perera et al. is significant for both practitioners and academics. For industry professionals, it provides a clear indication that DevOps adoption

can lead to significant quality improvement, which is essential in today’s rapidly changing software development environment. From an academic perspective, the study fills a significant gap in the literature by quantifying the impact of DevOps practices on software quality, providing a fundamental basis for further research in this emerging field.

Overall, the study “Improving Software Quality through DevOps Practices” is a valuable addition to the body of knowledge on software development methodologies, offering a solid understanding of the practical benefits of DevOps practices. [11]

1.2.5 ADOC conceptual model for automated DevSecOps

The literature review of Rakesh Kumar and Rinkaj Goyal’s paper “Modeling continuous security: A conceptual model for automated DevSecOps using open-source software over cloud (ADOC)” examines the integration of security into DevOps processes - referred to as DevSecOps - and addresses its inherent challenges using a conceptual security model enhanced with open source software (OSS) and cloud technologies. The study critically evaluates the trade-off between flexibility and security in software development and concludes that accelerated adoption of OSS and cloud solutions can significantly enhance security measures without compromising speed of delivery.

The ADOC conceptual model developed by the authors is innovative and presents a synthesis of a continuous security framework that spans the entire development and operations cycle. This model not only facilitates the automation of security controls, but also ensures that these controls are embedded in the DevOps workflow, thereby promoting a security engineering approach. The detailed description of the integration of OSS tools within this model is an example of a practical application that enhances both the security and operational efficiency of DevOps practices.

In addition, Kumar and Goyal’s study contributes to the existing literature by identifying specific challenges in DevSecOps implementation and proposing measurable solutions through their ADOC model. They emphasize the importance of metrics to evaluate the effectiveness of security integration in DevOps, which is critical for continuous improvement and risk management.

However, the study also recognizes limitations such as the variability of OSS quality and the need to continuously monitor and update security measures to combat evolving threats. This points to areas for further research, particularly standardizing the use of OSS and developing more robust mechanisms to manage the dynamic nature of cloud resources.

To summarize, this paper makes a significant contribution to the field by describing a structured approach to incorporating security into DevOps, leveraging the flexibility of OSS and the scalability of cloud technologies that

are critical to modern software development environments. This model serves as a blueprint for organizations seeking to improve their security protocols while maintaining or increasing speed. [12]

1.2.6 Comparative effectiveness and efficiency of DevOps and DevSecOps methodologies

The literature review of Jimmy Björnholm’s dissertation on “Performance of DevOps compared to DevSecOps: DevSecOps pipelines benchmarked!” is an academic analysis of the comparative effectiveness and efficiency of DevOps and DevSecOps methodologies, especially in the context of CI/CD pipeline performance. This study is based on a hands-on approach where the author integrated nine different security tools into a standard DevOps pipeline to evaluate their impact on build time, a critical metric in software development.

Björnholm uses the OWASP Top Ten and ISO/IEC 25010 standards as the basis for selecting and evaluating security tools, ensuring that his methodology is relevant and rigorous. This selection is not only in line with industry standards, but also provides a structured way to evaluate security in software pipelines. The practical setting of the experiments conducted using Microsoft Azure and OWASP Juice Shop as a test environment further strengthens the applicability of the study to real-world scenarios.

The results of the study are significant: integrating security measures into DevOps can increase build times by as little as one-third of the original duration, disproving the common perception that security significantly slows down development processes. This finding is particularly valuable for organizations seeking to balance fast development cycles with the need for robust security measures.

However, the study also recognizes its shortcomings, such as the potential for variability in results due to the limited number of runs of each tool and the specific configuration of the test environment. These factors may affect the generalizability of the results obtained and suggest directions for further research.

Overall, Björnholm’s dissertation contributes to the ongoing discussion on integrating security into DevOps practices by offering empirical data and detailed analysis that can assist practitioners and researchers in understanding and optimizing the performance of DevSecOps pipelines. This work highlights the feasibility and effectiveness of incorporating security early in the software development lifecycle without significantly compromising speed, advocating for a more security-conscious approach in CI/CD practices. [13]

1.3 Types of Security Testing

1.3.1 Static Application Security Testing (SAST)

In “Towards Better Utilizing Static Application Security Testing,” a study by Jinqiu Yang, Lin Tang, and John Peyton, presented at the 2019 IEEE/ACM 41st International Conference on Software Engineering, the authors examine the underutilization of static application security testing (SAST) by developers due to issues such as high false positive rates, excessive vulnerability warnings, and lack of actionable remediation guidance.

The study identifies three major challenges to the effective use of SAST: managing multiple vulnerability alerts, the prevalence of false positives, and the lack of actionable remediation suggestions. These challenges result in developers often ignoring the results of SAST, which could otherwise be a powerful tool for identifying and remediating software vulnerabilities.

To overcome these obstacles, Yang et al. present Priv, a tool designed to improve the usability of SAST through three areas: prioritizing vulnerability alerts based on common remediation locations, reducing false positives by eliminating SAST-specific bugs, and providing customized remediation suggestions. Priv’s innovative approach includes using visualization techniques to group and prioritize alerts and customize remediation suggestions to increase developer engagement and response efficiency.

The effectiveness of Priv is evaluated on a case study of six web applications showing significant improvements in identifying preferred fix locations, reducing false positives, and providing actionable remediation suggestions. The paper reports that Priv correctly identified preferred patch locations 75.3 percent of the time, matched them with annotations provided by security experts, and reduced false positives to zero for certain SAST cases.

This research has great practical implications for improving the effectiveness of SAST in software development. By addressing specific issues that reduce the effectiveness of SAST, the authors offer a path for developers to better interact with these tools, which ultimately improves the security and quality of software products. Moreover, the integration of Priv features into existing commercial products emphasizes its practical relevance and potential impact on industry practices.

The work of Yang et al. represents an important step forward in making SAST a more effective tool for developers by reducing noise and increasing the relevance of the results provided by these systems. This research not only contributes to academic knowledge, but also offers tangible solutions to improve security practices in software development environments. [14]

1.3.2 Comparing Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST)

Lyubka Dencheva's Master's thesis setup guide is a comprehensive study comparing Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) using open source web application penetration testing tools. This document serves as an important appendix to her study, detailing the experimental setup required to evaluate the effectiveness of these security testing methodologies in a controlled environment.

The guide carefully describes the setup of a Kali Linux virtual machine on Oracle VM VirtualBox, a critical step for creating a robust testing platform. Dencheva uses this setup to implement the SAST and DAST tools, providing a structured approach to installing, configuring, and executing these tools, which is critical to ensuring the validity and reliability of her research results.

Dencheva's selection of tools - from LGTM.COM and Visual Code Grepper for SAST to Arachni and OWASP ZAP for DAST - emphasizes the diversity and availability of open source resources in cybersecurity practice. Its methodical customization processes ensure that each tool is optimally configured to thoroughly validate the OWASP Benchmark, a standard testing scheme that provides a consistent, measurable basis for its analysis.

By juxtaposing the SAST and DAST methodologies, Dencheva's work addresses a significant gap in the literature on the practical application of these testing methods to detect and remediate vulnerabilities in software applications. The thorough documentation of each step in her setup guide not only enhances the reproducibility of her study, but also highlights the practical challenges and considerations involved in effectively deploying these tools.

In conclusion, Dencheva's configuration guide is a critical resource that supports her thesis by detailing the technical setup required to conduct a thorough comparative analysis of SAST and DAST. This guide not only complements her research, but also serves as a valuable resource for other researchers and practitioners interested in the field of cybersecurity, particularly application security testing. [15]

1.3.3 Robust Software composition analysis (SCA)

Philip Ombredann's article "Free and Open Source Software License Compliance: Tools for Analyzing Software Composition," published in IEEE's Computer magazine, addresses the complex landscape of license compliance in modern software development environments. The article explains the problems associated with managing the various licenses of the vast number of open source software components into which modern software products are integrated. Ombredann argues for the need to automate the management of these complexities due to the large number of components and their diverse license terms.

Ombredann emphasizes that much of software development now depends on the integration of pre-existing open source components, each with different origins, licensing, and dependencies. The fluid nature of open source software delivery, with frequent updates and changing licenses for each version, requires robust software composition analysis (SCA) tools to ensure compliance. It describes the critical role of these tools in identifying open source components in use, understanding their licensing terms, and ensuring compliance when those components are redistributed in commercial software products.

The article offers a deep dive into the various SCA methodologies, distinguishing between scanning and matching methods. Scanning involves directly extracting information from software files, primarily package manifests, license texts, and other provenance information. In contrast, matching involves comparing software components against a pre-indexed database of known open source components to identify similarities and establish provenance.

One of the main theses of Ombredann’s discussion is the inadequacy of current open source software documentation practices. He cites a study that found that less than 5 percent of the free and open source software packages analyzed contained clear and complete licensing documentation. This finding underscores the critical need for effective SCA tools that can fill these information gaps, ensuring that developers can fulfill their legal obligations when using open source software.

Ombredann then details the technical challenges and potential methodologies for implementing effective SCA, discussing tools such as Fossology, ScanCode Toolkit, and commercial offerings such as BlackDuck. He also touches on the potential of projects such as Software Heritage, which seeks to archive and index all publicly available software, providing an invaluable resource for compliance tools.

In conclusion, Ombredann’s article serves as a critical resource for understanding the software license compliance landscape in an era of ubiquitous open source software. His perspective on the challenges and necessary characteristics of effective SCA tools serves as a valuable guide for developers, project managers, and compliance professionals in this complex area. The discussion not only emphasizes the need for these tools, but also calls for continued innovation and development in this area to keep pace with the evolving nature of software development. [16]

1.3.4 Interactive application security testing (IAST)

The 2019 International Conference on Smart Grid and Electrical Automation (ICSGEA) paper, authored by Pan, presents a comprehensive review of advanced monitoring technologies for smart grids. The paper offers a detailed study of methodologies and technologies to improve the efficiency, reliability, and security

of smart grids, which are an integral part of modern power systems.

Pan’s research focuses on the various technologies underpinning smart grid operations, including the integration of renewable energy sources, the implementation of advanced metering infrastructure (AMI), and the application of real-time monitoring systems. The work emphasizes the critical role of these technologies in transforming traditional power systems into more adaptive, efficient, and responsive grids. The study of data analytics and machine learning algorithms is a significant contribution to the optimization of power systems and the management of the huge flow of data coming from smart grid components.

The methodological strength of the paper lies in the detailed analysis of real-time data processing techniques and the deployment of predictive models that improve decision-making processes in smart grids. Pan uses a combination of modeling and empirical data analysis to evaluate the performance of proposed solutions in different operational scenarios. This approach not only demonstrates the practical applicability of the research, but also emphasizes the potential for scalability in different power system environments.

The results of Pan’s study have profound implications for the future operation of power systems. The detailed analysis of smart grid technologies provides fundamental insights that can help in the design of more resilient and efficient power systems. In addition, the discussion of integration challenges and potential solutions makes a valuable contribution to overcoming barriers to the full integration of smart grids.

This paper makes a significant contribution to the academic and practical understanding of smart grid technologies. By providing a comprehensive overview of current technologies and future directions, Pan’s work serves as a valuable resource for researchers and practitioners in electrical engineering and smart grid technologies. The discussion of the implications of these technologies for regulatory policy and market operations further adds to the relevance of the study.

Overall, Pan’s paper at the 2019 ICSGEA conference is an important addition to the literature on smart grid technologies. It elaborates on the challenges and innovations in the field, offering both theoretical findings and practical solutions that pave the way for future advances in smart grid systems. The integration of theoretical research with practical applications makes this work a key reference for current developments in the deployment of smart grid technologies. [17]

1.4 Monitoring

1.4.1 Site reliability engineering (SRE)

Master thesis “Implementation of secure workflow for DevOps from best practices viewpoint” presents a comprehensive study of the integration of security practices into DevOps, called DevSecOps. The dissertation is structured to offer both theoretical and empirical perspectives on the implementation

of DevSecOps in organizations, especially focusing on strengthening security measures throughout the software development life cycle (SDLC).

In the literature review, Lamponen meticulously evaluates existing frameworks and methodologies that support the integration of security into DevOps practices. He discusses the evolution of DevSecOps and various key publications such as the “State of DevOps Report” and papers that critically analyze the transition from DevOps to DevSecOps. The review emphasizes the need for a security-centric approach in modern software development, discusses the challenges and proposes metrics to measure the effectiveness of DevSecOps.

The thesis provides a detailed account of the implementation of a DevSecOps workflow at Visma Solutions, where security is integrated into the pipeline from code commit to deployment in a production environment. Lamponen reviews various tools and technologies such as containerized applications and their security, static and dynamic application security testing, and software composition analysis. This case study serves to illustrate the application of the theoretical concepts discussed in the literature review.

This paper makes a significant contribution by bridging the gap between theoretical knowledge and practical implementation of DevSecOps. It provides practical insights into the configuration and optimization of security practices within DevOps, highlighting the importance of the tools and cultural shifts required for successful integration.

Lamponen concludes that a well-implemented DevSecOps workflow increases the frequency of implementations and correlates with higher organizational performance. The results of the empirical section support the theoretical claims by showing that integrating security into DevOps not only reduces risks but also increases operational efficiency.

Overall, Lamponen’s thesis makes a significant contribution to the understanding of DevSecOps by providing both a theoretical framework and empirical evidence to support the integration of security practices into software development workflows. [18]

1.4.2 Improving visibility in application security

Tuomas Tiensuu’s master’s thesis, “DevSecOps adoption: Improving visibility in application security” scrutinizes the literature to establish the importance of improving application security visibility as part of the DevSecOps implementation process. The thesis takes a detailed look at the prevalent methodologies of modern software development, emphasizing the integration of security at each stage of the software development life cycle (SDLC) to improve overall security visibility.

Tiensuu discusses the integration of security practices into DevOps, advocating a shift to DevSecOps where security is implemented from the beginning. The literature cited supports the idea that traditional DevOps practices, while effective

in accelerating development and deployment processes, often bypass security. Tiensuu uses platforms such as the NIST DevSecOps Framework and ISO 27001 to demonstrate established practices for aligning security with agile development methodologies.

The main argument of the thesis revolves around the critical importance of visibility in application security. Tiensuu argues that without comprehensive visibility into security processes and metrics, organizations will not be able to effectively manage security risks. The study highlights various tools and practices such as security information and event management (SIEM) systems and continuous integration/continuous deployment (CI/CD) pipelines that play an important role in providing such visibility.

Tiensuu’s empirical study, conducted through surveys and interviews in a large organization, provides insight into the current state of application security practices. The results show uneven integration of security across teams, emphasizing the need for a unified approach to security across the SDLC. These empirical findings are critical to validate the theoretical models discussed and provide a practical perspective on the challenges and benefits of DevSecOps implementation.

The thesis makes a significant contribution to the field by providing practical strategies for organizations to improve security by improving visibility. Tiensuu’s work goes beyond theoretical reasoning by providing practical solutions and frameworks that organizations can implement to ensure security as a central component of software development processes.

In conclusion, Tiensuu’s dissertation provides a comprehensive study of integrating security into DevOps practices, emphasizing the indispensable role of visibility in effectively managing application security. The reviewed literature not only justifies the need for a paradigm shift towards DevSecOps, but also offers a blueprint for achieving this transition in large organizations. [19]

1.4.3 Spatio-temporal data analysis

In the paper “Sustained emerging spatio-temporal co-occurrence pattern mining: A summary of results” presented at the 18th IEEE International Conference on Tools for Artificial Intelligence (ICTAI’06), the authors review the challenges and advances in finding steadily emerging spatio-temporal concurrence patterns (SECOPs). This literature review describes the key concepts, methodologies and contributions of this work, highlighting its importance in the field of spatio-temporal data analysis.

The research addresses the critical need to identify subsets of object types that co-occur in both space and time, which is important for various applications such as healthcare, military operations, and environmental monitoring. The authors emphasize the computational challenges posed by the sheer volume of data, the

complexity of measures of interest, and the exponential growth in the number of potential patterns. These challenges require efficient and scalable solutions to extract meaningful patterns from large spatio-temporal datasets.

Celik et al. propose a novel SECOP-Miner algorithm that introduces a monotonic composite measure of interest to help find SECOPs efficiently. This measure combines spatial and temporal prevalence to identify patterns that not only occur frequently, but also show increasing importance over time. The effectiveness of the algorithm is demonstrated through analytical and experimental validation and shows significant improvement over naive methods in terms of computational efficiency and accuracy.

The paper conducts a thorough evaluation of the proposed algorithm on real datasets, demonstrating its ability to handle large amounts of spatio-temporal data with different types of objects and time intervals. The authors provide a detailed performance analysis of the algorithm under different settings, which emphasizes the practical applicability of their approach in real-world scenarios. This evaluation is crucial for practitioners and researchers seeking to apply similar methods in their fields, especially in areas requiring fast and accurate detection of emerging patterns, such as disease outbreak monitoring and environmental studies.

This study makes a significant contribution to the literature by formalizing the concept of SECOPs and providing a robust methodological framework for their detection. Of particular note is the introduction of a monotonic measure of interest, which ensures not only the efficiency of the search process but also the preservation of all potentially useful patterns. This work lays the foundation for future research aimed at further optimizing and applying spatio-temporal cocurrency patterns.

To summarize, the work of Celik et al. offers a comprehensive solution to the challenging problem of spatio-temporal data analysis. The proposed SECOP-Miner algorithm and its associated measures of interest represent a significant achievement in the field, providing both theoretical insights and practical tools for finding complex patterns in large datasets. This study not only addresses the pressing problems of pattern detection in spatio-temporal data sets, but also lays the foundation for future research aimed at improving the scalability and applicability of search algorithms in various scientific and industrial domains. [20]

2 PRACTICAL PART

2.1 Closed infrastructure

To raise the entire CI/CD DevSecOps pipeline needed to first design and raise their secure infrastructure where they needed to deploy cloud native services. For the design, the free web service draw.io was used to outline the entire outline of the infrastructure on the DevSecOps Pipeline side, as well as the overall infrastructure itself. I used various examples of deploying such infrastructures based on examples from Google, Yandex.Cloud, IBM, etc.

2.1.1 Plan of infrastructure

After studying a lot of scientific articles and web resources, I came to the conclusion that I would first need to deploy 2 virtual servers based on Debian 12 with firewalls, web servers, firewall policies, and docker installed.

It was also necessary to automate the process of raising the basic configuration with the help of Ansible configuration orchestrator, so as not to spend too much time in the next times, if the project will be further migrated to other servers or someone wants to use the project for their own developments.

Besides, it was necessary to have a monitoring system to control the state of the server itself in case it starts slowing down and running out of resources during Back End API lifting with the help of Docker Compose, Gitlab repository, and local Gitlab runners, because they consume quite a lot of resources and can heavily load CPU and RAM.

Also, we should not forget that between the servers it will be necessary to provide secure HTTPS connection by creating self-signed certificates, so that attackers could not simply intercept the traffic and read it in the open, in cases if SSL certificates are not used to encrypt traffic within the entire infrastructure.

After taking all of these points into account, I began designing the topology itself using the draw.io web service I pointed out a bit above. Looking at the topology, it gradually projects a less clearer idea of the whole project, which will be raised here in the future.

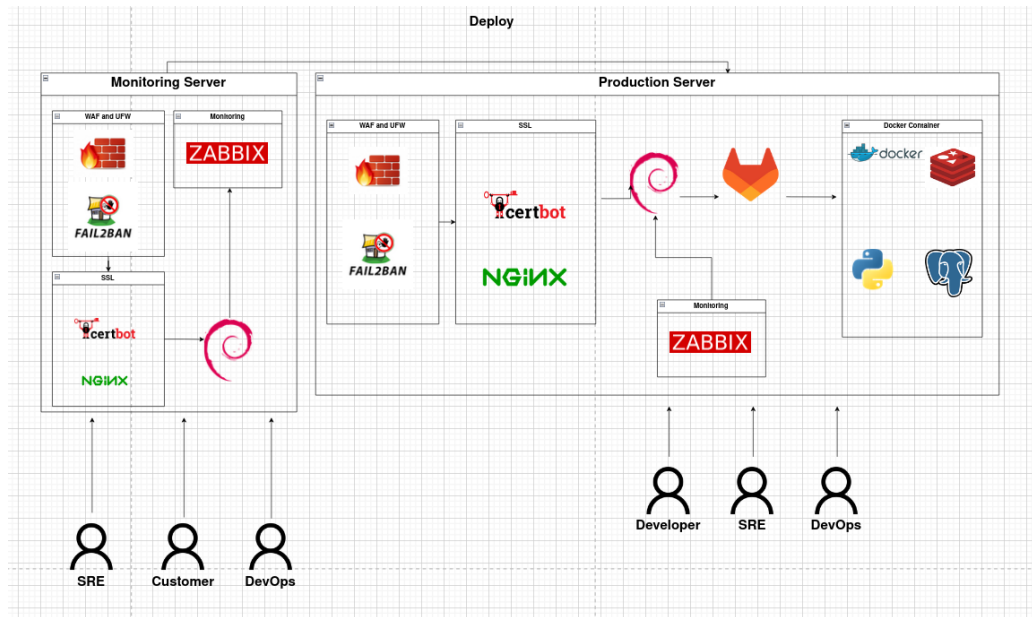


Figure 2.1 – Plan of infrastructure

As mentioned above, the closed infrastructure itself consists of two Debian 12 servers, one of which is the Production server, where the main part of the development environment will be located, and the second one will host the tools for accounting and controlling the state of the main Production server.

The Production server on the main port will host a local Gitlab repository, which will be used to store all possible runners, control access to them, and automate the process of building applications by using Gitlab Runners for Pipeline.

It will also host the Back End API service, which will be based on Python and the FastAPI web development framework. For the database, it is planned to deploy a Redis cached NoSQL database for temporary information that needs quick access, as well as a PostgreSQL SQL database for the main data.

Before the services themselves it is planned to deploy Nginx in reverse-proxy mode with installed self-signed SSL certificates for redirection on the main port to the Gitlab repository, as well as on an additional port to the Back End API application. To provide security on the server itself against various dangerous activities, a UFW firewall will be configured with basic policies to open ports for specific ip addresses.

For more complex policies on redirecting and rejecting any dangerous requests, attempts to scan with various scanners to detect vulnerabilities, attempts to spoof SSL certificates, attempts to get to paths in the application that do not exist, Fail2Ban with additional proprietary filters was used.

For monitoring, a Zabbix agent will be installed, which collects metrics on the state of swap partitions, physical server partitions, RAM usage, CPU load, user password changes, etc. on the Linux server and will send them to the Monitoring

server for collection

The Monitoring server will also host a Nginx server with the same policies that were configured on the previous Production server. It will also have a Zabbix Server deployed on it through the use of Ansible. Zabbix server has its own convenient separate admin panel, dashboards, sent alerts in Telegram, and many other things that will make it easier to monitor the status of the Production server.

2.1.2 Using hypervisor VMWare ESXi for creating VMs

This part of the thesis will now describe the use of VMWare ESXi server for raising virtual machines themselves. VMWare ESXi is a 1st order hypervisor, which provides on the physical server to create high-performance virtual machines, VLANs between them, virtual switches, and much more.

First, you will need to log into VMware ESXi server by its ip address and enter credits for authorization. Beforehand you will need to download the official Debian 12 image from the official website and upload it to the server's local storage so that you can boot from this ISO file during the installation. After authorization you will need to go to the menu with virtual machines and click on create virtual machine. There we choose Debian 12 (64-bit) operating system and proceed to customize the characteristics of our server. Here we can select the number of CPUs, RAM, hard disk size, video memory, as well as network adapters that will be used to connect to the Internet, etc.

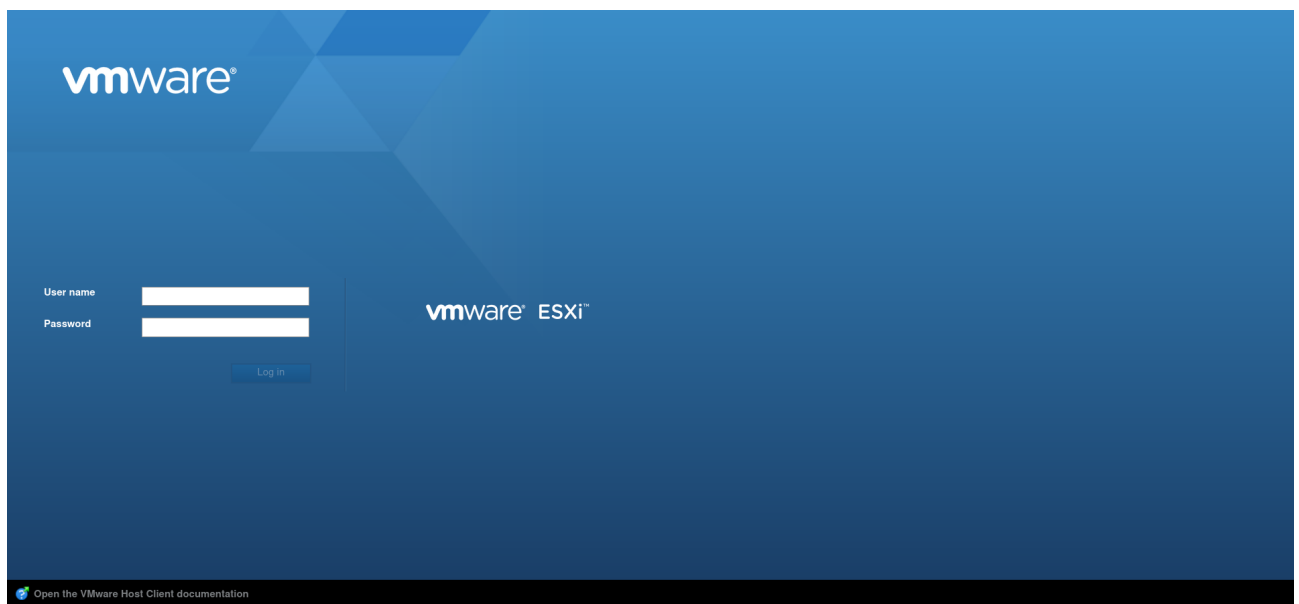


Figure 2.2 – VMWare Login

First, I create a Production server, which will need at least 4gb of RAM and 2 CPUs, since the local Gitlab repository is a pretty large-sized service that consists of a bunch of other small ones, which can consume a lot of resources. I allocate at least 100GB of disk space.

After that I create the virtual machine and basic installation, where I create LVM in case I run out of free space on the physical partition to expand it by reformatting the size of the hard disk connected to the virtual machine.

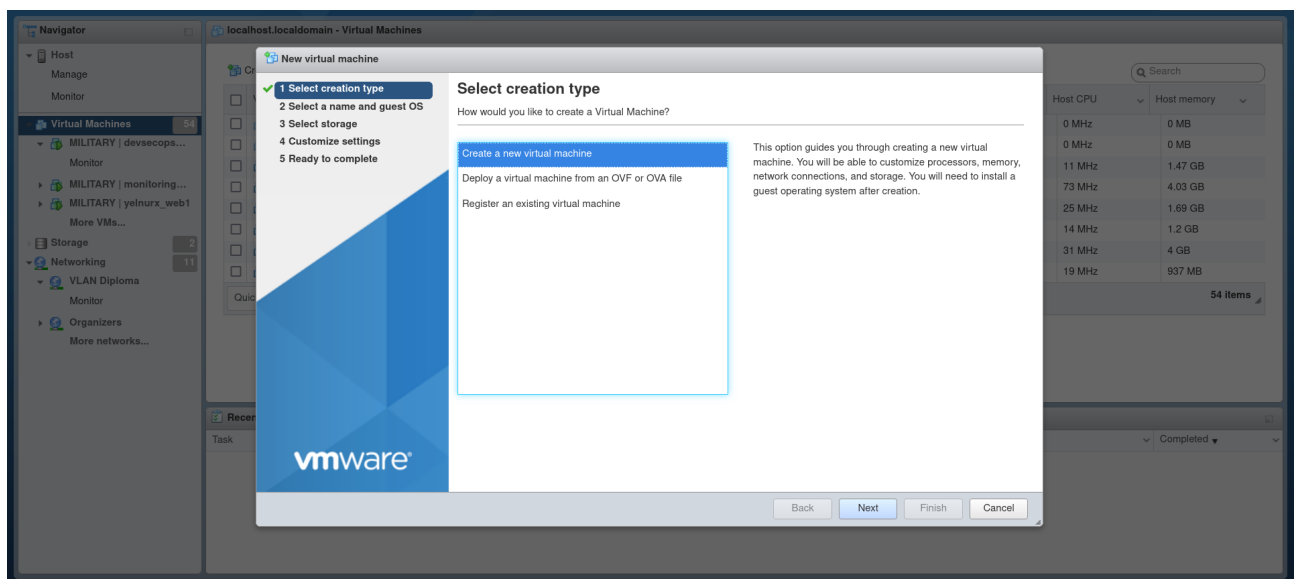


Figure 2.3 – Create VM - Step 1

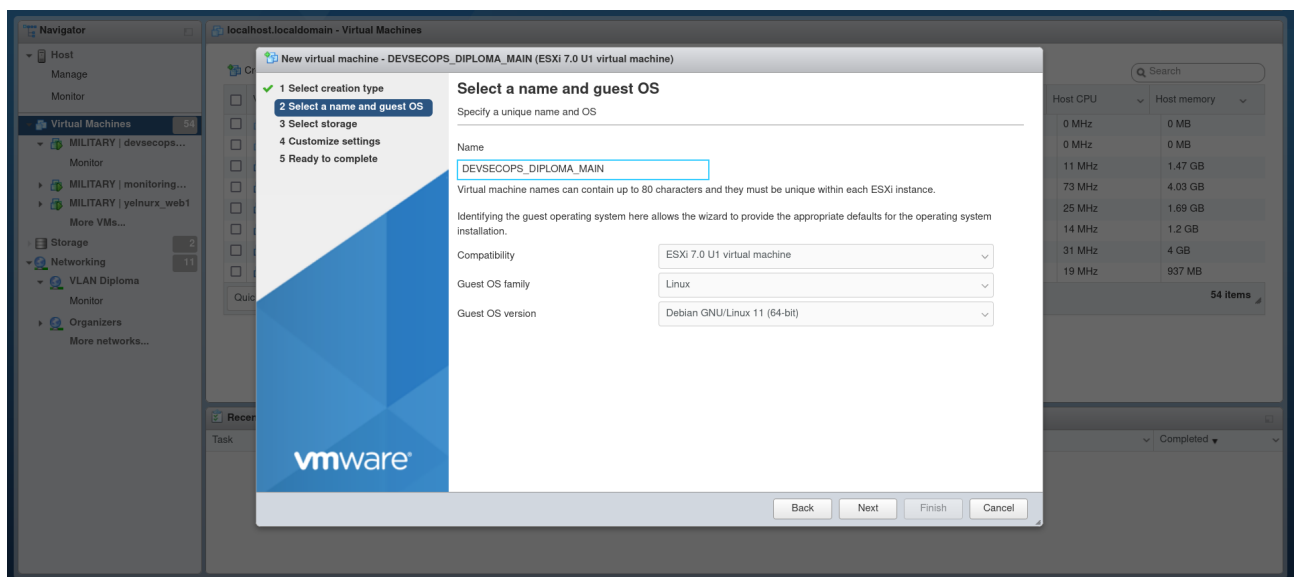


Figure 2.4 – Create VM - Step 2

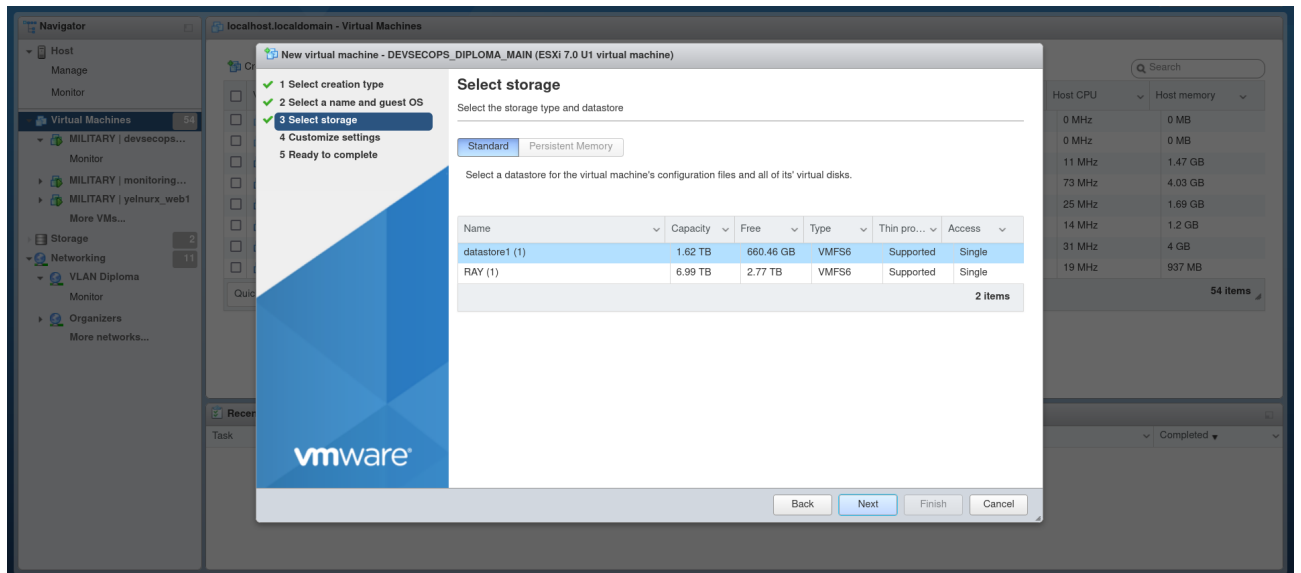


Figure 2.5 – Create VM - Step 3

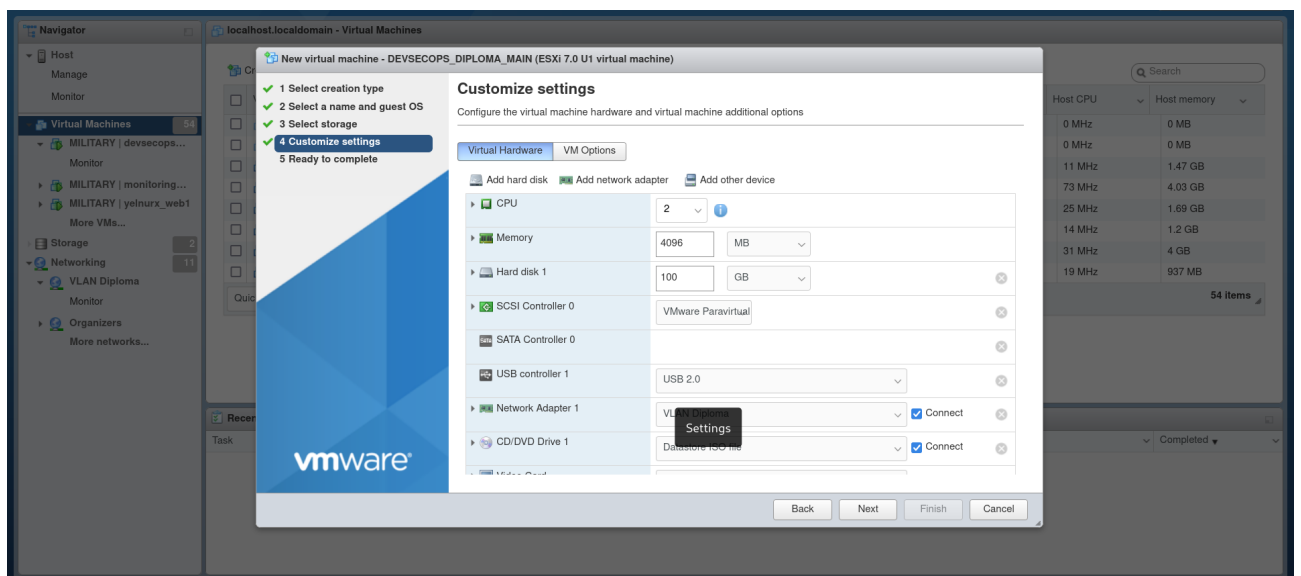


Figure 2.6 – Create VM - Step 4

The monitoring server is similar, but there I allocate 1 CPU and 2 GB RAM, because Zabbix Server is not a heavy service like Gitlab Repository. Also LVM was used for partitioning, as there can be a situation where log files can clog up the entire hard disk on the server.

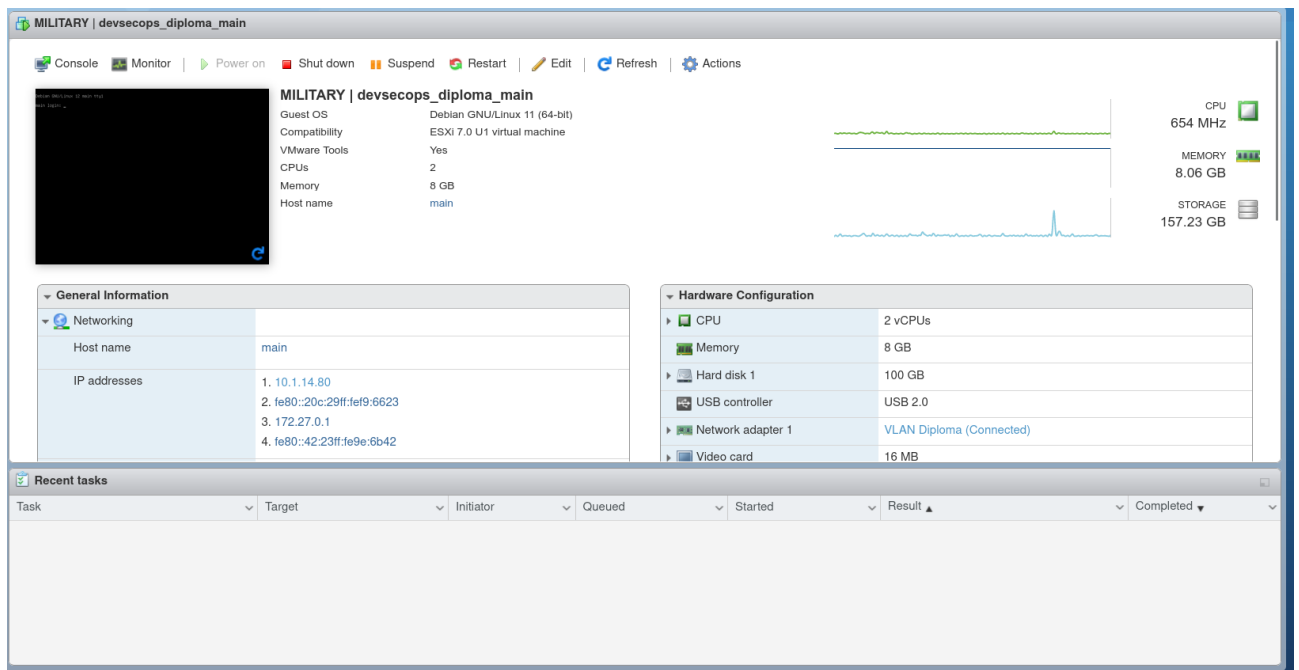


Figure 2.7 – Production Server in VMWare ESXi

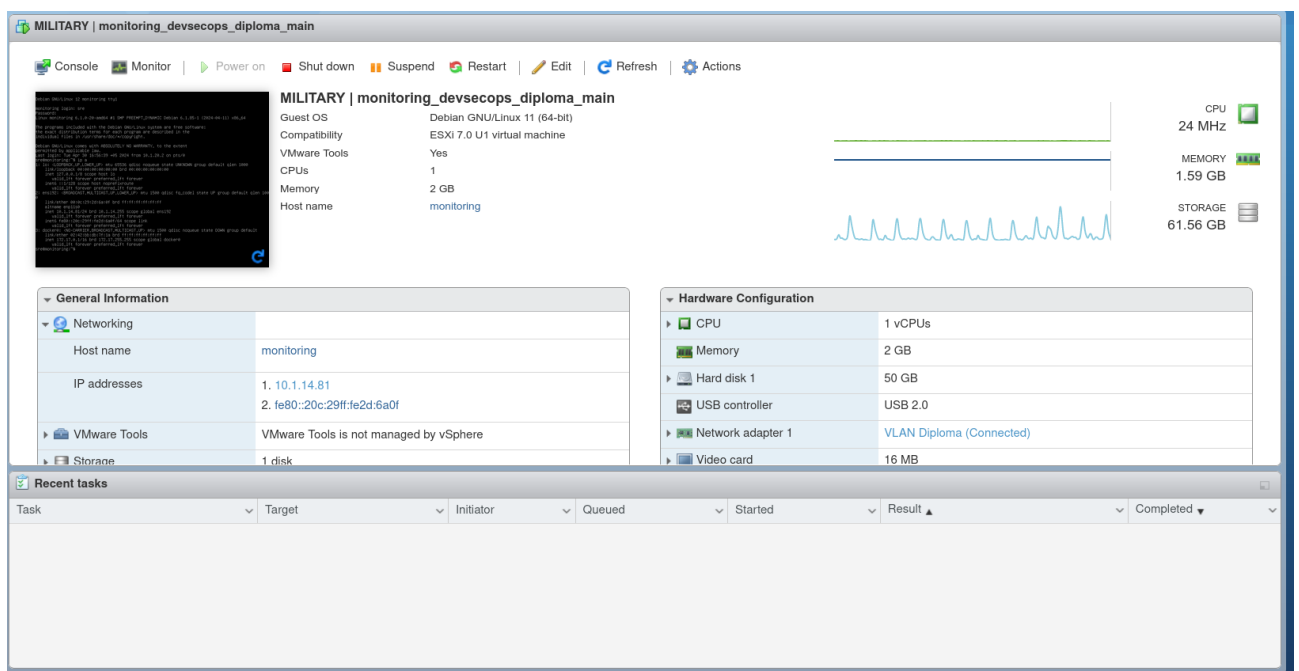


Figure 2.8 – Monitoring Server in VMWare ESXi

2.1.3 Autoconfiguration of VMs with Ansible

The next task was to do a completely basic configuration of the servers using the Ansible configuration orchestrator. First we had to write a list of actions that it had to perform before moving on to writing the configuration file itself. Tasks to be automated by this script:

- 1 Install basic logging packages for the server.
- 2 Installing additional utilities for server administration

- 3 Firewall configuration
- 4 Installing and configuring Fail2ban policies
- 5 Connecting Docker repository and Docker Compose for build projects
- 6 Configuring Nginx web server
- 7 Generating self-signed SSL certificates and adding them to Nginx.

The file consists of the main `my-install-script.yaml` inventory and others that will be used if the conditions are met. If the server operating system is Debian-based, the special `debian.yaml` inventory from Appendix A 1.5 for Debian is used, and there are also conditions for the inventory for RHEL-based operating systems.

The `debian.yaml` inventory from Appendix A 1.5 performs the first 4 tasks mentioned above. First, it installs the packages `rsyslog`, `python3-pip`, `python3-cryptography`, `logrotate`, `vim`, `curl`, `htop`, `neofetch`, `net-tools`, `ufw`, `nginx` to perform the first two tasks. After that it does Firewall configuration, which accomplishes task number 3. Then at the end of the day, he configures Fail2ban, where he takes and copies configuration files from a remote repository.

The `docker-debian-install.yaml` inventory from Appendix A 1.6 performs task number 5, where it takes and plugs in the official Docker repository to download and install dependencies for building Docker images.

First it removes the old version of Docker and its dependencies if they were installed. Then it installs the `curl` and `ca-certificates` packages to download the repository key. The next step is to change the permissions inside the `/etc/keyrings` directory so that all users have limited access to the keys and cannot modify them unlike the root user. After that, the script downloads the APT key, changes its permissions and adds it to the correct directory.

The script then adds the repository to the `/etc/apt/sources.d/` folder and installs Docker itself. If the script is successfully executed, the handlers are triggered and the downloaded services are enabled for Docker to function.

The `nginx-debian-install.yaml` inventory from Appendix A 1.7 performs tasks 6 and 7, in which Nginx should already be configured and self-signed SSL certificates should be created and installed.

First, it installs the `nginx` packages for the web server itself, as well as the `openssl` package for creating SSL keys. After that, it uses the Ansible `community.crypto` module to generate a public and private key to secure the HTTPS connection. Finally, the default configuration file for Nginx is copied from the remote repository and if this task is successful, the service is restarted and auto-start is enabled

```

[main]~/VSCodiumProjects/linux-ansible-configuration]$ ansible-playbook my-install-script.yaml

PLAY [Test playbook for Web Server] *****

TASK [Gathering Facts] *****
ok: [192.168.124.206]

TASK [Debian based server detection] *****
included: /home/devsexops/VSCodiumProjects/linux-ansible-configuration/debian.yaml for 192.168.124.206 => (item=debian.yaml)
included: /home/devsexops/VSCodiumProjects/linux-ansible-configuration/docker-debian-install.yaml for 192.168.124.206 => (item=docker-debian-install.yaml)
included: /home/devsexops/VSCodiumProjects/linux-ansible-configuration/nginx-debian-install.yaml for 192.168.124.206 => (item=nginx-debian-install.yaml)

TASK [Debian based server detection] *****
ok: [192.168.124.206] => {
  "msg": "Finding Debian-based server..."
}

```

Figure 2.9 – Detect OS version

```

TASK [Default packages install] *****
ok: [192.168.124.206]

TASK [Configure UFW to allow SSH] *****
ok: [192.168.124.206]

TASK [Configure UFW to allow HTTP] *****
ok: [192.168.124.206]

TASK [Configure UFW to allow HTTPS] *****
ok: [192.168.124.206]

```

Figure 2.10 – Package installation with UFW Configuration

```

TASK [Fail2ban install] *****
ok: [192.168.124.206]

TASK [Add special filters for Fail2ban] *****
ok: [192.168.124.206]

TASK [Configure Fail2ban jail.local] *****
ok: [192.168.124.206]

```

Figure 2.11 – Fail2ban Configuration

```

TASK [Install Docker Engine] *****
ok: [192.168.124.206] => {
  "msg": "Starting install Docker Engine..."
}

TASK [Uninstall old versions of dependencies] *****
ok: [192.168.124.206]

TASK [Debian dependencies install] *****
ok: [192.168.124.206]

TASK [Set permissions for /etc/apt/keyrings] *****
ok: [192.168.124.206]

TASK [Add Docker APT key] *****
ok: [192.168.124.206]

TASK [Set permission for APT key] *****
changed: [192.168.124.206]

TASK [Add Docker APT repository] *****
ok: [192.168.124.206]

TASK [Install Docker Engine dependencies] *****
ok: [192.168.124.206]

```

Figure 2.12 – Docker Repository connection with installation

```

TASK [Nginx configuration] *****
ok: [192.168.124.206] => {
  "msg": "Starting Nginx Configuration..."
}

TASK [Nginx install] *****
ok: [192.168.124.206]

TASK [Create self-signed private key for Nginx] *****
ok: [192.168.124.206]

TASK [Create self-signed cert for Nginx] *****
ok: [192.168.124.206]

TASK [Configure Nginx default.conf] *****
ok: [192.168.124.206]

TASK [RHEL based server detection] *****
skipping: [192.168.124.206]

PLAY RECAP *****
192.168.124.206      : ok=25   changed=1    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0

```

Figure 2.13 – Nginx configuration with SSL-certificates generation

2.1.4 Set up of local Gitlab Repository

To install the local Gitlab repository, we used the instructions from the official Gitlab documentation. First, the dependencies of the service had to be installed with the command “`sudo apt-get install -y curl openssh-server ca-certificates perl`”. After that, you need to download the official Gitlab installer, which will add their official repository and install Gitlab using the command “`curl https://packages.gitlab.com/install/repositories/gitlab/gitlab-ee/script.deb.sh | sudo bash`”.

Once the installation has been done, we need to change the default port of Gitlab from 443 to another port, as 443 will host the external Nginx, which will redirect traffic to the Back End API on port 8001 and Gitlab on port 443. Once these steps have been done, we already run the “`gitlab-ctl reconfigure`” command to rebuild Gitlab and make the default port 10003.

Once this has been done, we will need to rewrite the Nginx configuration file to enable reverse-proxy mode on it. This configuration is shown in Figure 2.15.

After the Nginx configuration was finished, I log into the Gitlab web panel using the server ip address and enter the default password to change the password, create users and upload global repositories inside the local one. An example of the authorization web panel is shown in Figure 2.16.

After authorization, I imported the repository inside Gitlab using a .tar.gz archive and the Import Project button (Figure 2.17) on the web panel. I imported the repository for the Back End API service (Figure 2.18)

```
# nginx['ssl_dhparam'] = nil # Path to dhparams.pem, eg. /etc/gitlab/ssl/dhparams.pem
# nginx['ssl_password_file'] = nil # Path to file with passphrases for ssl certificate secret keys
# nginx['listen_addresses'] = ['*', '[:]:']

##! **Defaults to forcing web browsers to always communicate using only HTTPS**
##! Docs: https://docs.gitlab.com/omnibus/settings/nginx.html#setting-http-strict-transport-security
# nginx['hsts_max_age'] = 63072000
# nginx['hsts_include_subdomains'] = false

##! Defaults to stripping path information when making cross-origin requests
# nginx['referrer_policy'] = 'strict-origin-when-cross-origin'

##! **Docs: http://nginx.org/en/docs/http/nginx_http_gzip_module.html**
# nginx['gzip_enabled'] = true

##! **Override only if you use a reverse proxy**
##! Docs: https://docs.gitlab.com/omnibus/settings/nginx.html#setting-the-nginx-listen-port
nginx['listen_port'] = 10003

##! **Override only if your reverse proxy internally communicates over HTTP**
##! Docs: https://docs.gitlab.com/omnibus/settings/nginx.html#supporting-proxied-ssl
nginx['listen_https'] = false

##! **Override only if you use a reverse proxy with proxy protocol enabled**
##! Docs: https://docs.gitlab.com/omnibus/settings/nginx.html#configuring-proxy-protocol
# nginx['proxy_protocol'] = false
```

Figure 2.14 – Change of default port for Gitlab

```

server {
    listen 80;
    server_name 10.1.14.80;

    location / {
        proxy_pass http://127.0.0.1:10003;
        proxy_set_header Host $http_host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}

server {
    listen 443 ssl;
    server_name 10.1.14.80;

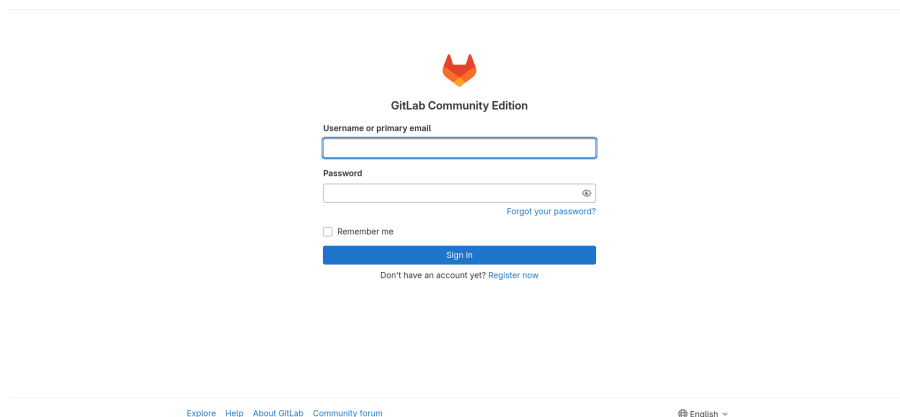
    ssl_certificate /etc/ssl/certs/nginx-selfsigned.crt;
    ssl_certificate_key /etc/ssl/private/nginx-selfsigned.key;

    location / {
        proxy_pass http://127.0.0.1:10003;
        proxy_set_header Host $http_host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}

server {
    listen 8001;
    server_name 10.1.14.80;
    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $http_host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}

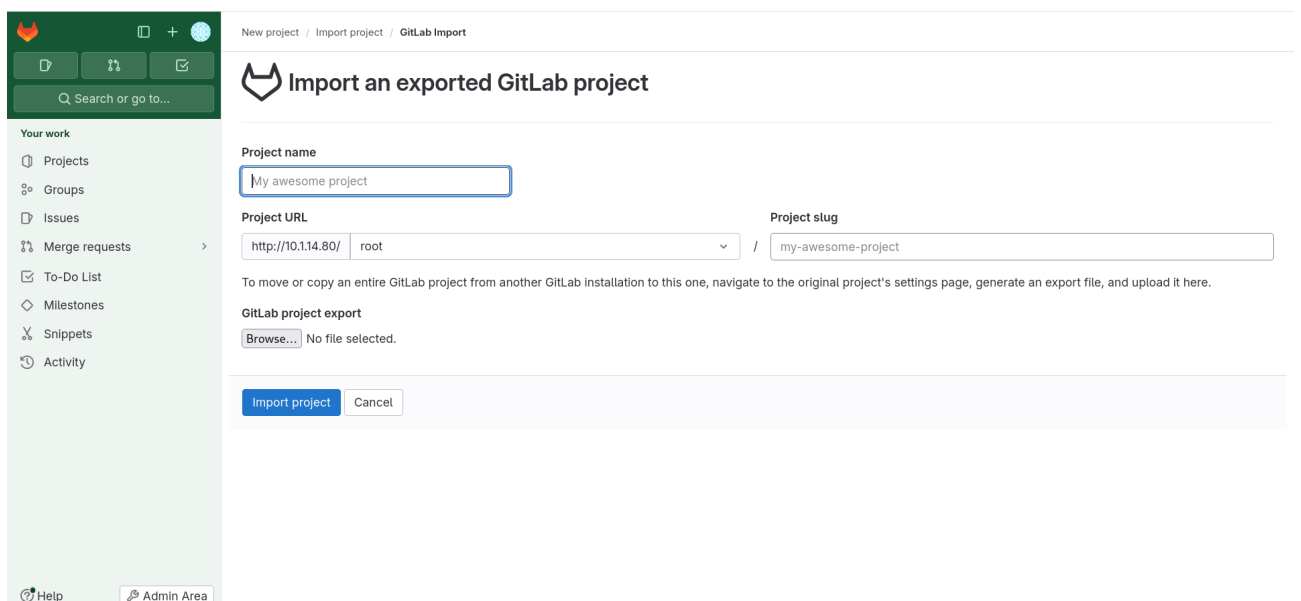
```

Figure 2.15 – Change of Nginx configuration



The image shows the GitLab Community Edition login page. At the top center is the GitLab logo (an orange fox head). Below it, the text "GitLab Community Edition" is displayed. The login form includes a text input for "Username or primary email", a password input with a toggle for visibility, and a "Remember me" checkbox. A "Sign in" button is positioned below the password field. A link for "Forgot your password?" is located to the right of the password input. At the bottom of the form, there is a link that says "Don't have an account yet? Register now". The footer of the page contains navigation links: "Explore", "Help", "About GitLab", and "Community forum", along with a language selector set to "English".

Figure 2.16 – Gitlab Authorization page



The image displays the GitLab "Import an exported GitLab project" page. On the left is a sidebar with the GitLab logo and navigation options: "Your work", "Projects", "Groups", "Issues", "Merge requests", "To-Do List", "Milestones", "Snippets", and "Activity". At the bottom of the sidebar are links for "Help" and "Admin Area". The main content area has a breadcrumb trail: "New project / Import project / GitLab Import". The title "Import an exported GitLab project" is followed by a form. The "Project name" field contains "My awesome project". The "Project URL" field is split into two parts: "http://10.1.14.80/" and "root". The "Project slug" field contains "my-awesome-project". Below these fields, a note states: "To move or copy an entire GitLab project from another GitLab installation to this one, navigate to the original project's settings page, generate an export file, and upload it here." Under the heading "GitLab project export", there is a "Browse..." button and the text "No file selected." At the bottom of the form are two buttons: "Import project" and "Cancel".

Figure 2.17 – Gitlab Import Project Function

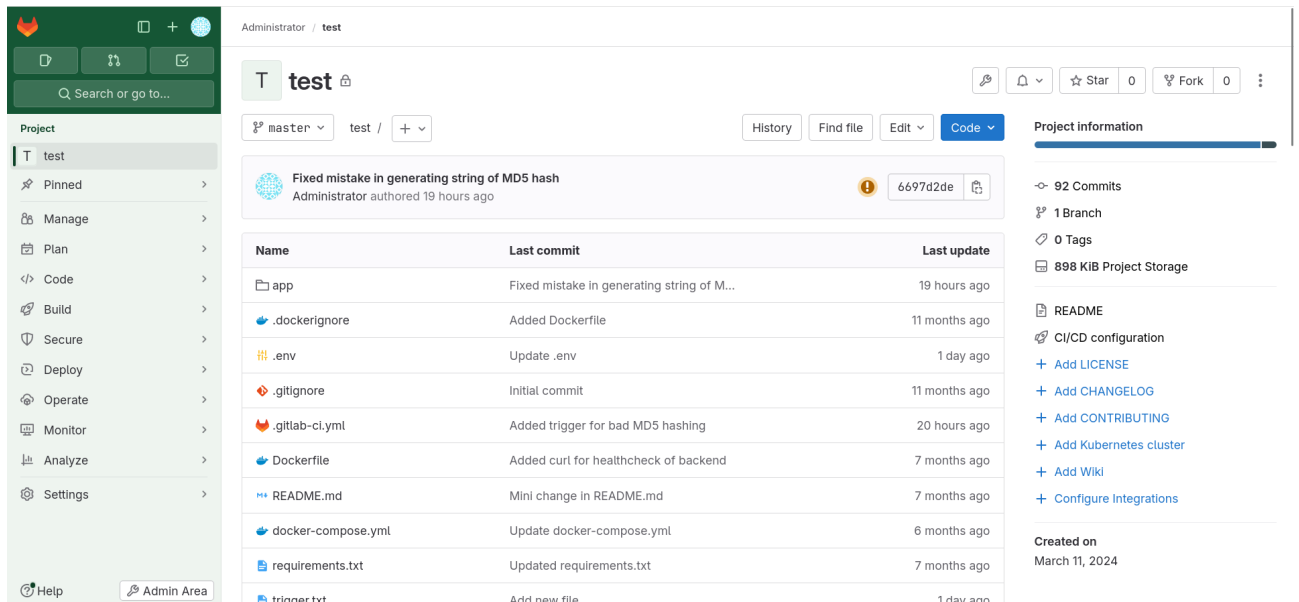


Figure 2.18 – Back End API Repository

2.1.5 Set up of local Gitlab Runners with Docker and Shell executors

Now we need to install Gitlab executors to automate the application build processor. In this thesis I installed two different kinds of executors.

The first executor I installed was a Shell executor that works like a systemd service. All commands it executes are executed within the host machine itself. In this job it was used to run git pull on the server to update the local repository, and also to run a docker compose file to trigger container rebuilds while making changes to the repository. The installation is very similar to the Gitlab repository installation, where you just download the installation file and run it as a service. After installation, you need to register it through the Gitlab Web panel by clicking the Register Gitlab Runner button (Figure 2.19).

The second Executor I installed was the Docker executor. It executes all commands inside its Docker environment, which allows you to run tools inside Docker containers after which they are immediately removed as soon as any task is completed. It has been mostly used to work with DevSecOps tools, as many of them have ready built images inside Docker containers and templates to run in Gitlab Pipelines. Installation is done with one button through the Docker image build and also by registering the Gitlab Runner (Figure 2.19)

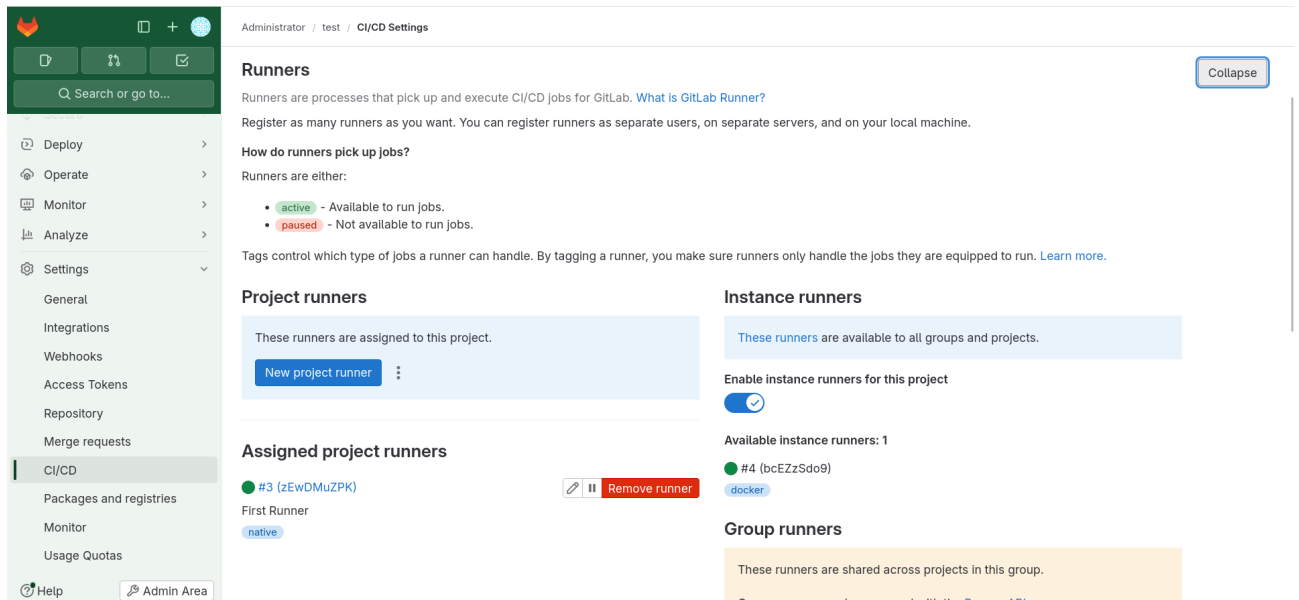


Figure 2.19 – Registration of Gitlab Runner

2.2 Back End API

After lifting Gitlaba, it was necessary to write a simple Back End API service whose development environment will be protected later. In this project, I wrote a simple API in the Python programming language using the FastAPI web framework (Figure 2.20).

The API itself has two basic endpoints (Appendix B 2.1.4), one of which can simply register a user (Figure 2.21) and the second of which can list all registered users within the API service (Figure 2.22).

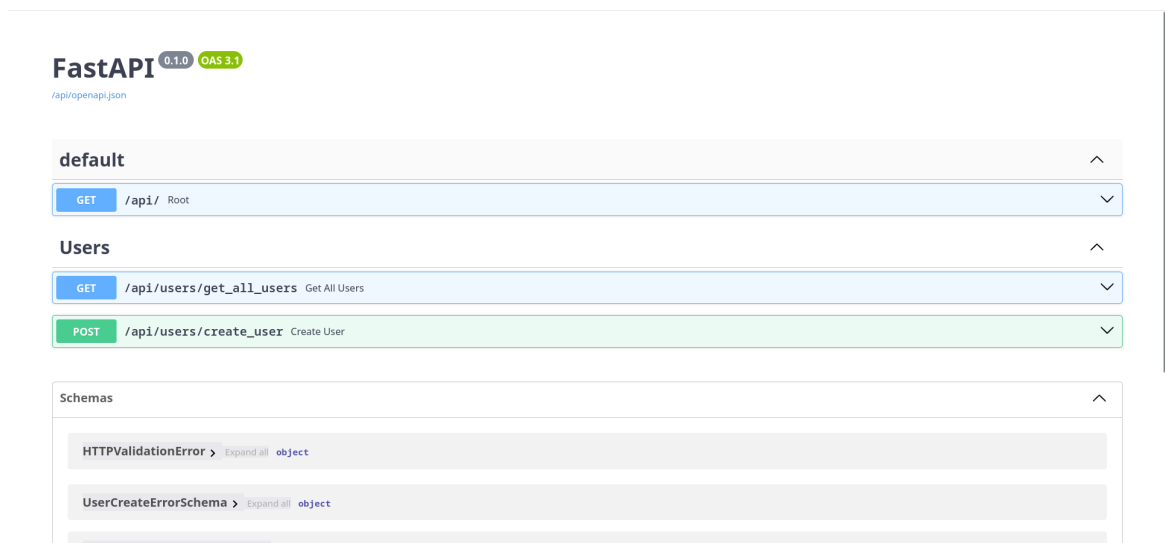


Figure 2.20 – Swagger of FastAPI Back End

After the service itself was written, a vulnerability with MD5 hashing (Appendix B 2.1.4) was deliberately added into the code logic to provoke analyzers during its building. Since MD5 has long been an obsolete hashing method and is easily decrypted at the moment

Then a Dockerfile (Appendix B 2.4) was created to build a Docker image of this Back End application to run inside the container. Finally, the databases that used this API service were also added inside the docker containers and their full autorun was configured via the docker-compose.yml file (Appendix B 2.5)

POST /api/users/create_user Create User

Parameters

No parameters

Request body **required**

application/json

Example Value | Schema

```
{
  "name": "Abay",
  "surname": "Sadykov",
  "password": "12345678",
  "email": "pythondeloy@gmail.com"
}
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type: application/json

Controls Accept header:

Figure 2.21 – Create user Endpoint

GET /api/users/get_all_users Get All Users

Parameters

No parameters

Responses

Code	Description	Links
200	Successful Response	No links
500	Problems with DB	No links

Media type: application/json

Controls Accept header:

Example Value | Schema

```
{
  "success": true,
  "msg": [
    {}
  ]
}
```

Figure 2.22 – Get all users Endpoint

2.3 Zabbix Monitoring

Now the monitoring stage has begun, where after working with Production-server we need to configure Monitoring-server to remotely know the current server load without manual connection via SSH.

2.3.1 Install and configuration of Zabbix Server

To install Zabbix server, unlike Gitlab, more steps are needed, because some services need to be manually lifted beforehand. Zabbix consists of the application itself written in PHP/GO, Nginx and SQL database. Nginx and Zabbix itself is installed by the installer, but the PostgreSQL database and its users have to be created manually.

Also, after Zabbix has been installed, you will need to enter the configuration file `/etc/zabbix/zabbix-server.conf` (Figure 2.23) and configure the port on which the Zabbix server will be located, as well as enter the password for the PostgreSQL database (Figure 2.24), so that Zabbix will have the right to make entries there.

Once all the above actions have been done, you need to go to the Zabbix admin panel (Figure 2.25) and enter the default password “zabbix” to get into the web panel and configure hosts and users.

```
server {
    listen 443 ssl;
    ssl_certificate /etc/ssl/certs/nginx-selfsigned.crt;
    ssl_certificate_key /etc/ssl/private/nginx-selfsigned.key;

    root /usr/share/zabbix;

    index index.php;

    location = /favicon.ico {
        log_not_found off;
    }

    location / {
        try_files $uri $uri/ =404;
    }

    location /assets {
        access_log off;
        expires 10d;
    }

    location ~ /\.ht {
        deny all;
    }

    location ~ /(api|conf|include|locale) {
        deny all;
        return 404;
    }

    location /vendor {
        deny all;
        return 404;
    }

    location ~ [\?]\.php(/|$) {
        fastcgi_pass unix:/var/run/php/zabbix.sock;
        fastcgi_split_path_info ^(.+\.php)(/.+)$;
        fastcgi_index index.php;
    }
}
```

Figure 2.23 – Nginx configuration of Zabbix Server

```

DBName=zabbix
## Option: DBSchema
#   Schema name. Used for PostgreSQL.
#
# Mandatory: no
# Default:
# DBSchema=

## Option: DBUser
#   Database user.
#
# Mandatory: no
# Default:
# DBUser=

DBUser=zabbix

## Option: DBPassword
#   Database password.
#   Comment this line if no password is used.
#
# Mandatory: no
# Default:
DBPassword=place_for_password

## Option: DBSocket
#   Path to MySQL socket.
#
# Mandatory: no
# Default:
# DBSocket=

## Option: DBPort
#   Database port when not using local socket.
#   If the Net Service Name connection method is used to connect to Oracle database, the port number from the
#   tnsnames.ora file will be used. The port number set here will be ignored.
#
# Mandatory: no
# Range: 1024-65535
-- INSERT --

```

Figure 2.24 – DBPassword in zabbix_server.conf for PostgreSQL using

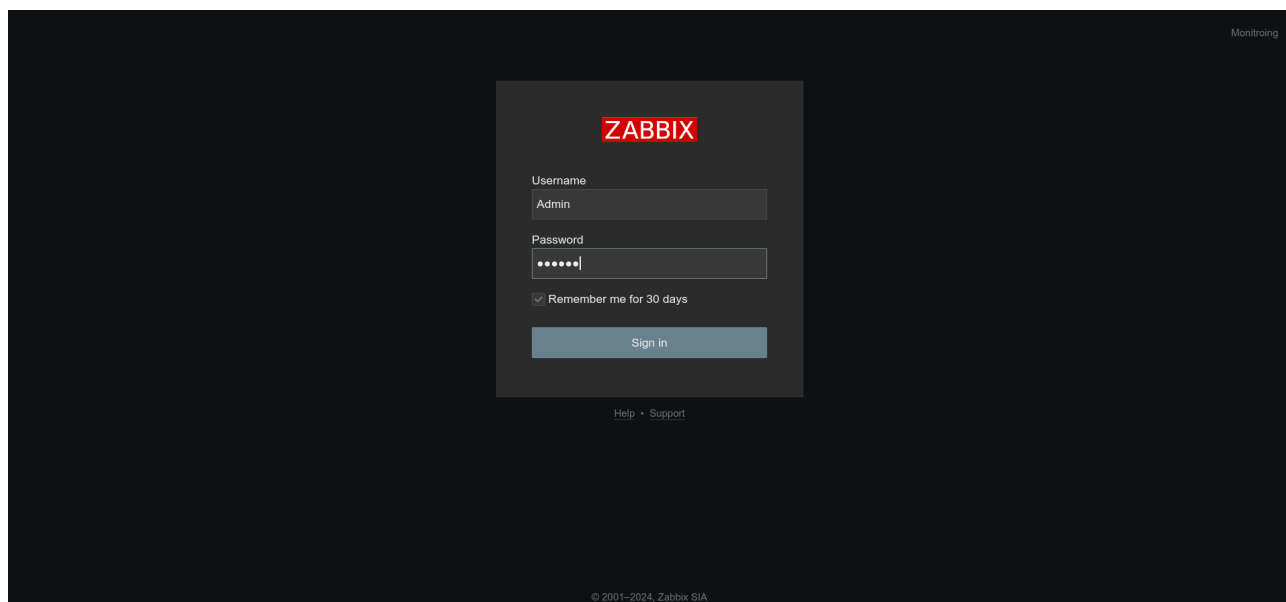


Figure 2.25 – Zabbix Authorization page

2.3.2 Install and configuration of Zabbix Agent

Installing Zabbix Agent is similar to installing Zabbix server and is much faster and easier, because you don't need to create any database on the analyzed server.

All you need to do after the basic installation of the deb package is to go to the file `/etc/zabbix/zabbix_agent.conf` and add the ip address of our Zabbix server to the `Server` and `ServerActive` variables (Figure 2.26, Figure 2.27) so that Zabbix Agent will allow our Zabbix server to collect data.

```
ServerActive=127.0.0.1, 10.1.14.81

## Option: Hostname
# List of comma delimited unique, case sensitive hostnames.
# Required for active checks and must match hostnames as configured on the server.
# Value is acquired from HostnameItem if undefined.
#
# Mandatory: no
# Default:
# Hostname=
Hostname=Gitlab

## Option: HostnameItem
# Item used for generating Hostname if it is undefined. Ignored if Hostname is defined.
# Does not support UserParameters or aliases.
#
# Mandatory: no
# Default:
# HostnameItem=system.hostname
```

Figure 2.26 – ServerActive configuration

```
Server=127.0.0.1, 10.1.14.81

## Option: ListenPort
# Agent will listen on this port for connections from the server.
#
# Mandatory: no
# Range: 1024-32767
# Default:
# ListenPort=10050

## Option: ListenIP
# List of comma delimited IP addresses that the agent should listen on.
# First IP address is sent to Zabbix server if connecting to it to retrieve list of active checks.
#
# Mandatory: no
# Default:
# ListenIP=0.0.0.0

## Option: StatusPort
# Agent will listen on this port for HTTP status requests.
#
```

Figure 2.27 – Server configuration

2.3.3 Adding of Telegram Alerts

After the Zabbix monitoring setup was finished, I added Telegram notifications that are sent to me in real time via a regular Telegram bot. An example of its messages is below (Figure 2.28).

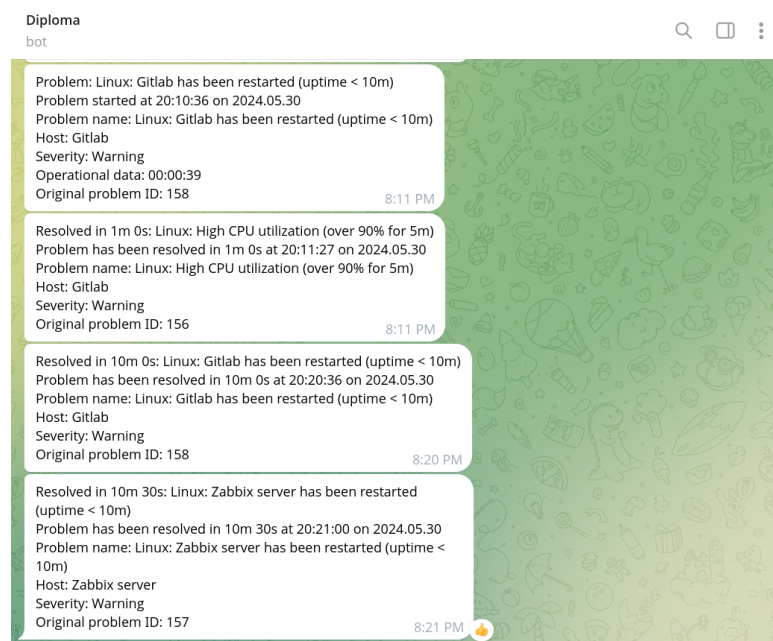


Figure 2.28 – Telegram alerts from Zabbix

2.4 Gitlab DevSecOps CI/CD Pipeline

Now after a very large part of time has been spent on raising the most closed infrastructure in which development will go, we start integrating the use of DevSecOps in CI/CD pipelines by combining different analyzers and running them with shell/docker executors. The Pipeline topology (Figure 2.29) consists of 4 main steps:

- 1 Secret Detection, which will analyze the history of git and the commits within it for various secrets
- 2 Build, where the project will be built and instantly launched.
- 3 Static Application Security Testing (SAST), where the whole code is analyzed for obsolete functions and methods.
- 4 Software Composition Analysis (SCA), which collects information about all used dependencies and possible import errors in the code.

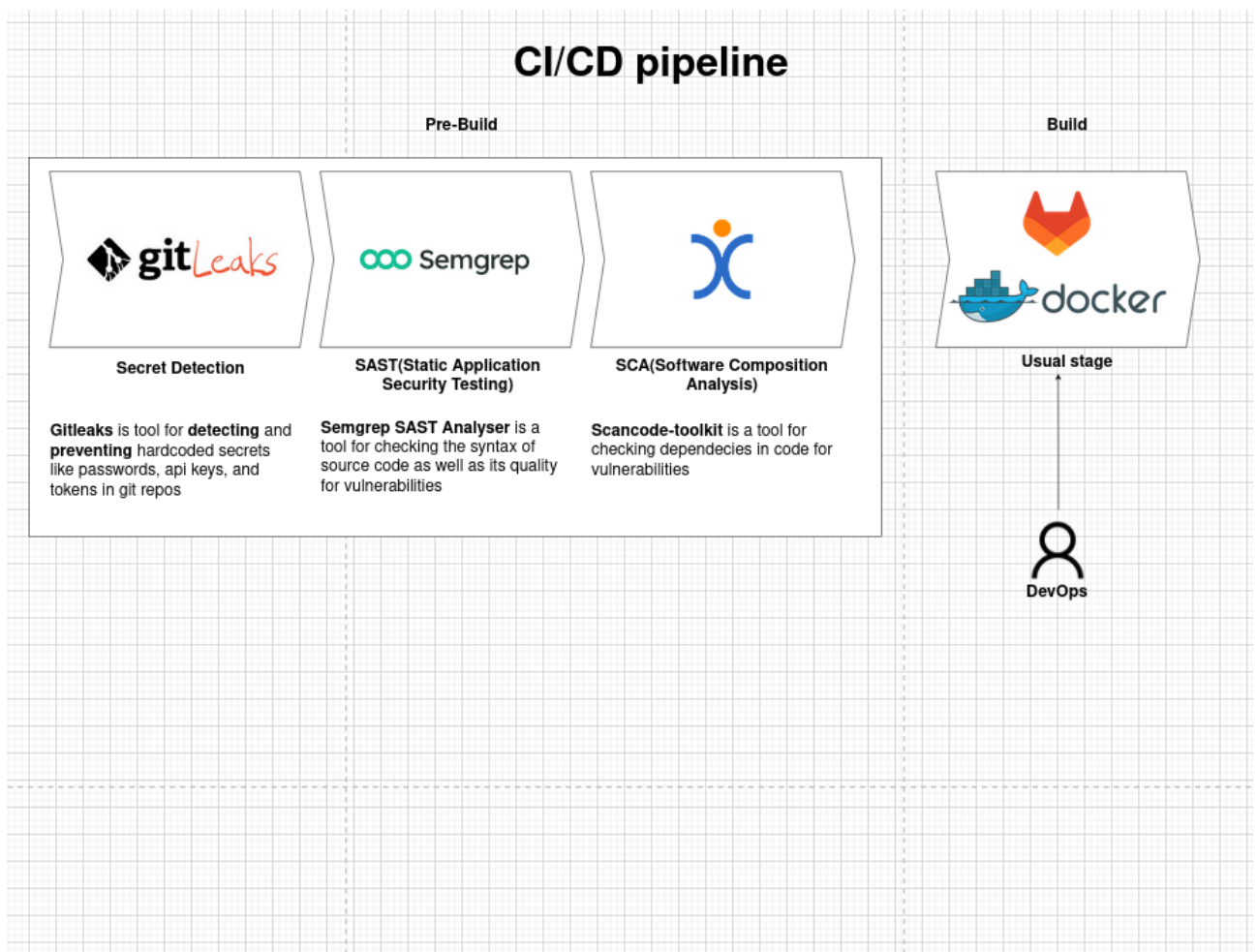


Figure 2.29 – DevSecOps Pipeline

2.4.1 Secret Detection

To detect secrets, a docker container with Git-leaks wrapped inside it is used. It comes at the very first stage before the application itself is built (Appendix C 3.1), because building an application with a secret in the git history is the worst security error.

The docker executor is used to run this analyzer. Below is an example where I deliberately put a private RSA key inside my repository (Figure 2.31) to provoke the analyzer to stop building the application (Figure 2.30).

2.4.2 Build

This step is a part of any regular DevOps Pipeline. These actions are performed on the host itself by the shell executor. It performs a git pull to get the latest changes inside the repository, and runs docker compose build (Figure 2.32) to build the application itself (Appendix C 3.1).

Figure 2.30 – Leak detection

```
{
  "version": "15.1.0",
  "vulnerabilities": [
    {
      "id": "df442a5c49f6294df8da3ab4b7881bad88f226ef8ac58a8c6fcf6e3d3d3e3438",
      "category": "secret_detection",
      "name": "RSA private key",
      "description": "RSA private key secret has been found in commit 5c615fed.",
      "cve": "trigger.txt:8bcac7988eb958419537b91e19adc83ce2c9cbfdacf4f81157fdadfec11f7017:RSA private key",
      "severity": "Critical",
      "confidence": "Unknown",
      "raw_source_code_extract": "-----BEGIN RSA PRIVATE KEY-----",
      "scanner": {
        "id": "gitleaks",
        "name": "gitleaks",
        "location": {
          "file": "trigger.txt",
          "commit": {
            "author": "Administrator",
            "date": "2024-06-02T06:53:28Z",
            "message": "Add new file",
            "sha": "5c615fed5cbdb3de63d921d8686d46a8ce542861",
            "start_line": 1
          },
          "identifiers": [
            {
              "type": "gitleaks_rule_id",
              "name": "gitleaks rule ID RSA private key",
              "value": "RSA private key"
            }
          ],
          "scan": {
            "analyzer": {
              "id": "secrets",
              "name": "secrets",
              "url": "https://gitlab.com/gitlab-org/security-products/analyzers/secrets",
              "vendor": {
                "name": "gitlab",
                "version": "8.18.2"
              },
              "type": "secret_detection",
              "start_time": "2024-06-02T07:01:23",
              "end_time": "2024-06-02T07:01:23",
              "status": "success"
            }
          }
        }
      }
    ]
  }
}
```

Figure 2.31 – Git-leaks report

```

14 $ git checkout master
15 Previous HEAD position was 6697d2d Fixed mistake in generating string of MD5 hash
16 Switched to branch 'master'
17 Your branch is behind 'origin/master' by 1 commit, and can be fast-forwarded.
18   (use "git pull" to update your local branch)
19 $ git pull
20 Updating ef676c8..6697d2d
21 Fast-forward
22  app/routers/v1/user_router.py | 2 +-
23  1 file changed, 1 insertion(+), 1 deletion(-)
24 $ docker compose up -d --build
25 time="2024-06-02T18:28:31+05:00" level=warning msg="/home/gitlab-runner/builds/zEwDMuZPK/0/root/test/docker-
compose.yml: 'version' is obsolete"
26 #0 building with "default" instance using docker driver
27 #1 [fastapi internal] load build definition from Dockerfile
28 #1 transferring dockerfile: 609B done
29 #1 DONE 0.0s
30 #2 [fastapi internal] load metadata for docker.io/library/python:3.10-slim-bullseye
31 #2 DONE 11.2s
32 #3 [fastapi internal] load .dockerignore
33 #3 transferring context: 1.66kB done
34 #3 DONE 0.0s
35 #4 [fastapi 1/8] FROM docker.io/library/python:3.10-slim-bullseye@sha256:9745f361fffc367922210f2de48a58f4478
2ebf5a7375195e91ebd5b3ce5a8ff
36 #4 DONE 0.0s
37 #5 [fastapi internal] load build context
38 #5 transferring context: 72.42kB 0.0s done
39 #5 DONE 0.0s
40 #6 [fastapi 3/8] RUN apt-get -y install curl

```

Figure 2.32 – Automatic build of project

2.4.3 Static Application Security Testing (SAST)

Semgrep is used for static code analysis for vulnerabilities also inside a docker container. It is executed with the help of Gitlab docker executor.

This analyzer takes and reads all the code sources until it finds any errors in the logic of the application itself or the use of dangerous obsolete methods for which various CVEs were found long ago (Figure 2.33). As an intentional vulnerability, I used the obsolete MD5 hashing method for passwords instead of SHA256. An example of this vulnerability detection is shown in Figure 2.34

```

16 Downloading artifacts for build_project (206)...
17 Downloading artifacts from coordinator... ok      host=10.114.80 id=206 responseStatus=200 OK token=glcb
t-64
18 Executing "step_script" stage of the job script
19 Using docker image sha256:c21f9602615ff52705ed62142dbec08554e7a302ae63d5500d19443edb289f6 for registry.gitl
ab.com/security-products/semgrep:5 with digest registry.gitlab.com/security-products/semgrep@sha256:761127ca
16dd083389b64dd08b4892df4020b0ebfa284d6810db2c4782546d4cf ...
20 $ /analyzer run
21 [INFO] [Semgrep] [2024-06-02T13:28:51Z] ▶ Gitlab Semgrep analyzer v5.2.2
22 [INFO] [Semgrep] [2024-06-02T13:28:51Z] ▶ Detecting project
23 [INFO] [Semgrep] [2024-06-02T13:28:51Z] ▶ Analyzer will attempt to analyze all projects in the repository
24 [INFO] [Semgrep] [2024-06-02T13:28:51Z] ▶ Running analyzer
25 [WARN] [Semgrep] [2024-06-02T13:28:51Z] ▶ /builds/root/test/.gitlab/sast-ruleset.toml not found, ruleset sup
port will be disabled.
26 [INFO] [Semgrep] [2024-06-02T13:29:04Z] ▶ Creating report
27 [WARN] [Semgrep] [2024-06-02T13:29:04Z] ▶ .gitlab/sast-ruleset.toml not found, ruleset support will be disab
led.
28 Uploading artifacts for successful job
29 Uploading artifacts...
30 gl-sast-report.json: found 1 matching artifact files and directories
31 Uploading artifacts as "sast" to coordinator... 201 Created id=207 responseStatus=201 Created token=glc6t-6
4
32 Job succeeded

```

Figure 2.33 – Generating of Semgrep report

```
{
  "version": "15.0.7",
  "vulnerabilities": [
    {
      "id": "9ba4df3345aaa3f0f66767e31b1512ac01f0693d2574ef0d8fc6ef5a3ac7c8e2",
      "category": "sast",
      "name": "Use of a broken or risky cryptographic algorithm",
      "description": "The application was found using an insecure or risky digest or signature algorithm. MD2, MD4, MD5 and SHA1 hash algorithms have been found to be vulnerable to producing collisions. This means that two different values, when hashed, can lead to the same hash value. If the application is trying to use these hash methods for storing passwords, then it is recommended to switch to a password hashing algorithm such as Argon2id or PBKDF2. Note that the 'crypto' and 'cryptodome' Python packages are no longer recommended for new applications, instead consider using the 'cryptography' (https://cryptography.io/) package. Example of creating a SHA-384 hash using the 'cryptography' package: from cryptography.hazmat.primitives import hashes create a SHA384 digest pdigest = hashes.Hash(hashes.SHA384()) Update the digest with some initial data pdigest.update(b'some data to hash') Add more data to the digest pdigest.update(b'some more data') Finalize the digest as bytes result = digest.finalize() For more information on secure password storage see OWASP: https://cheatsheetsseries-owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html For more information on the cryptography module see: https://cryptography.io/en/latest/ CVE: semgrep_id: bandit.B383-1:34:34, severity: Medium, scanner: {id: semgrep, name: semgrep, location: {file: app/routers/v1/user_router.py, start_line: 34}, identifiers: [{type: semgrep_id, name: bandit.B383-1, value: bandit.B383-1, ucl: https://semgrep.dev/r/gitlab-bandit.B383-1}, {type: cwe, name: CWE-327, value: 327, ucl: https://cwe.mitre.org/data/definitions/327.html}], {type: owasp, name: A02:2021 - Cryptographic Failures, value: A02:2021}, {type: semgrep, name: A3:2017 - Sensitive Data Exposure, value: A3:2017}, {type: bandit_test_id, name: Bandit Test ID B383, value: B383}], dependency_files: [], scan: {analyzer: {id: semgrep, name: semgrep, ucl: https://gitlab.com/gitlab-org/security-products/analyzers/semgrep, vendor: {name: Gitlab, version: 5.2.2}, scanner: {id: semgrep, name: semgrep, ucl: https://github.com/returntocorp/semgrep, vendor: {name: Gitlab, version: 1.68.0}, type: sast, start_time: 2024-06-02T13:28:51, end_time: 2024-06-02T13:29:04, status: success}}
    }
  ]
}
```

Figure 2.34 – Semgrep SAST report

2.4.4 Software Composition Analysis (SCA)

For simple analysis of dependencies inside the project and possible incorrect imports, the opensource analyzer scancode-toolkit installed natively on the server itself is used. It is already launched through shell executor as the final stage of analyzing the whole code. An example of this analyzer's report is below.

```
hexa/scancode-toolkit/ for support and download.",
  "start_timestamp": "2024-06-02T13:28:56.989178",
  "end_timestamp": "2024-06-02T13:29:10.333423",
  "output_format_version": "3.1.0",
  "duration": 3.3442633152088857,
  "message": null,
  "errors": [],
  "warnings": [],
  "extra_data": {
    "system_environment": {
      "operating_system": "Linux",
      "cpu_architecture": "64",
      "platform": "Linux-6.1.0-20-amd64-x86_64-with-glibc2.36",
      "platform_version": "#1 SMP PREEMPT_DYNAMIC Debian 6.1.85-1 (2024-04-11)",
      "python_version": "3.11.2 (main, Mar 13 2023, 12:18:29) [GCC 12.2.0]"
    },
    "spdx_license_list_version": "3.23",
    "files_count": 33
  }
},
"packages": [],
"dependencies": [
  {
    "purl": "pkg:pypi/pydantic@2.4.2",
    "extracted_requirement": "pydantic==2.4.2",
    "scope": "install",
    "is_runtime": true,
    "is_optional": false,
    "is_resolved": true,
    "resolved_package": {},
    "extra_data": {
      "is_editable": false,
      "link": null,
      "hash_options": [],
      "is_constraint": false,
    }
  }
]
```

Figure 2.35 – Scancode-toolkit report

CONCLUSION

Results acquired:

- Theoretical explorations covered the aims and importance of DevSecOps, different types of security testing
- Comparative analysis highlighted the efficiency and effectiveness of DevOps versus DevSecOps methodologies
- The infrastructure was set up using hypervisor VMWare ESXi, with autoconfiguration managed by Ansible.
- The setup included a local GitLab repository and runners configured with Docker and Shell executors.
- Monitoring was implemented via Zabbix, with specific configurations for the server, agents, and Telegram alerts.
- A comprehensive GitLab DevSecOps CI/CD pipeline was established, featuring secret detection, build processes, and security testing stages like SAST and Software Composition Analysis (SCA).
- The practical implementation demonstrated the feasibility and benefits of a well-orchestrated DevSecOps approach, paving the way for more secure and robust software development practices.

Improvements that can be made:

- Add a Dynamic Application Security Testing in CI/CD Pipeline by using OWASP ZAP benchmark
- Integrate a more powerful free or paid dependency analyzer inside the project
- Set up Vulnerabilities Dashboard based on OWASP Defect Dojo for security analysers

BIBLIOGRAPHY

- 1 *Izurieta, Clemente*. Leveraging SecDevOps to Tackle the Technical Debt Associated with Cybersecurity Attack Tactics / Clemente Izurieta, Mary Prouty // 2019 IEEE/ACM International Conference on Technical Debt (TechDebt). — 2019. — Pp. 33–37.
- 2 *Jammeh, Bakary*. DevSecOps: Security Expertise a Key to Automated Testing in CI/CD Pipeline / Bakary Jammeh // *Bournemouth University*. — 2020.
- 3 *Sánchez-Gordón, Mary*. Security as culture: a systematic literature review of DevSecOps / Mary Sánchez-Gordón, Ricardo Colomo-Palacios // Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops. — 2020. — Pp. 266–269.
- 4 Federal Cloud & Infrastructure Summit Report / Justin F Brunelle, Cameron Boozarjomehri, David Hansen et al. — The MITRE Corporation. Retrieved from [https://atarc.org/wp-content/uploads ...](https://atarc.org/wp-content/uploads...), 2019.
- 5 *Carter, Kim*. Francois raynaud on DevSecOps / Kim Carter // *IEEE Software*. — 2017. — Vol. 34, no. 5. — Pp. 93–96.
- 6 *Carturan, Sara*. Major Challenges of Systems-of-Systems with Cloud and DevOps—a financial experience report / Sara Carturan, Denise Goya // 2019 IEEE/ACM 7th International Workshop on Software Engineering for Systems-of-Systems (SESoS) and 13th Workshop on Distributed Software Development, Software Ecosystems and Systems-of-Systems (WDES) / IEEE. — 2019. — Pp. 10–17.
- 7 *Mansfield-Devine, Steve*. DevOps: finding room for security / Steve Mansfield-Devine // *Network security*. — 2018. — Vol. 2018, no. 7. — Pp. 15–20.
- 8 *McGraw, Gary*. Silver Bullet Talks with Tanya Janca / Gary McGraw // *IEEE Security & Privacy*. — 2018. — Vol. 16, no. 05. — Pp. 7–11.
- 9 FEDERAL DEVOPS SUMMIT / Michelle Casagni, MITRE Lead, Diane Hanf et al.
- 10 *Deck, Rebecca*. Adapting AppSec to a DevOps World. — 2019.
- 11 *Perera, Pulasthi*. Improve software quality through practicing DevOps / Pulasthi Perera, Roshali Silva, Indika Perera // 2017 Seventeenth International Conference on Advances in ICT for Emerging Regions (ICTer) / IEEE. — 2017. — Pp. 1–6.
- 12 *Kumar, Rakesh*. Modeling continuous security: A conceptual model for automated DevSecOps using open-source software over cloud (ADOC) / Rakesh Kumar, Rinkaj Goyal // *Computers & Security*. — 2020. — Vol. 97. — P. 101967.
- 13 *Björnholm, Jimmy*. Performance of DevOps compared to DevSecOps:

DevSecOps pipelines benchmarked! — 2020.

14 Towards better utilizing static application security testing / Jinqiu Yang, Lin Tan, John Peyton, Kristofer A Duer // 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP) / IEEE. — 2019. — Pp. 51–60.

15 *Dencheva, Lyubka*. Comparative analysis of Static application security testing (SAST) and Dynamic application security testing (DAST) by using open-source web application penetration testing tools: Ph.D. thesis / Dublin, National College of Ireland. — 2022.

16 *Ombredanne, Philippe*. Free and open source software license compliance: Tools for software composition analysis / Philippe Ombredanne // *Computer*. — 2020. — Vol. 53, no. 10. — Pp. 105–109.

17 *Pan, Yuanyuan*. Interactive application security testing / Yuanyuan Pan // 2019 International Conference on Smart Grid and Electrical Automation (ICSGEA) / IEEE. — 2019. — Pp. 558–561.

18 *Lamponen, Niclas*. Implementation of secure workflow for DevOps from best practices viewpoint / Niclas Lamponen. — 2021.

19 *Tiensuu, Tuomas*. DevSecOps adoption: Improving visibility in application security / Tuomas Tiensuu. — 2022.

20 Sustained emerging spatio-temporal co-occurrence pattern mining: A summary of results / Mete Celik, Shashi Shekhar, James P Rogers, James A Shine // 2006 18th IEEE International Conference on Tools with Artificial Intelligence (ICTAI'06) / IEEE. — 2006. — Pp. 106–115.

Appendix A Ansible Configuration Files

1.1 default.conf

HTTP Configuration

```
server {  
    listen 80;  
    return 301 https://$host$request_uri;  
}
```

HTTPS Configuration

```
server {  
    listen 443 ssl;  
    ssl_certificate /etc/ssl/certs/nginx-selfsigned.crt;  
    ssl_certificate_key /etc/ssl/private/nginx-selfsigned.key;
```

Port forwarding

```
# location / {  
# proxy_pass http://localhost:number_of_port;  
# proxy_set_header X-Real-IP $remote_addr;  
#     proxy_set_header X-Forwarded-Host $host;  
#     proxy_set_header x-Forwarded-Port $server_port;  
# }
```

Include MIME Types

```
include /etc/nginx/mime.types;
```

```
}
```

1.2 jail.local

[DEFAULT]

banaction = ufw

backend = auto

ignoreip =

[sshd]

enabled = true

port = ssh

filtered = sshd

logpath = /var/log/auth.log

findtime = 3600

maxretry = 6

```
bantime = 86400
```

```
[nginx-http-auth]
enabled = true
port    = http,https
filter  = nginx-http-auth
logpath = %(nginx_error_log)s
```

```
[nginx-botsearch]
enabled = true
port    = http,https
filter  = nginx-botsearch
logpath = %(nginx_access_log)s
```

```
[nginx-4xx]
enabled = true
port    = http,https
filter  = nginx-4xx
logpath = %(nginx_error_log)s
```

```
[nginx-forbidden]
enabled = true
port    = http,https
filter  = nginx-forbidden
logpath = %(nginx_error_log)s
```

```
[nginx-sslerror]
enabled = true
port    = http,https
filter  = nginx-sslerror
logpath = %(nginx_error_log)s
```

```
[nginx-limit-req]
```

```
enabled = true
port    = http,https
filter  = nginx-limit-req
action  = iptables-multiport[name=http, port="http,https", protocol=tcp
logpath = %(nginx_access_log)s
findtime = 600
```

```
maxretry = 100
bantime = 3600
```

```
[nginx-badbots]
```

```
enabled = true
port    = http,https
filter  = nginx-badbots
logpath = %(nginx_access_log)s
maxretry = 1
findtime = 604800
bantime  = 604800
```

1.3 filter.d

1.3.1 nginx-badbots.conf

```
# Create as file: /etc/fail2ban/filter.d/nginx-badbots.conf
[Definition]
```

```
badbots = 360Spider|404checker|404enemy|80legs|Abonti|Aboundex|Acuneti
```

```
failregex = (?i)<HOST> -.*"(GET|POST|HEAD).*HTTP.*(?:%(badbots)s).*"
```

```
ignoreregex =
```

1.4 my-install-script.yaml

```
---
```

```
- name: Test playbook for Web Server
  hosts: test
  tasks:
    # Debian-based server Configuration
    - name: Debian based server detection
      ansible.builtin.include_tasks: "{{ item }}"
      loop:
        - debian.yaml
        - docker-debian-install.yaml
        - nginx-debian-install.yaml
      when: ansible_os_family == 'Debian'

    # RHEL-based server Configuration
    - name: RHEL based server detection
```

```

    ansible.builtin.include_tasks: redhat.yaml
    when: ansible_os_family in ['RedHat', 'Rocky']
handlers:
  - name: Import Debian handlers
    ansible.builtin.import_tasks: debian-handlers.yaml
    when: ansible_os_family == 'Debian'

```

1.5 debian.yaml

```

---
- name: Debian based server detection
  debug:
    msg: Finding Debian-based server...

- name: Default packages install
  become: true
  apt:
    state: present
    name:
      - rsyslog
      - python3-pip
      - python3-cryptography
      - logrotate
      - vim
      - curl
      - htop
      - neofetch
      - net-tools
      - ufw
      - nginx

- name: Configure UFW to allow SSH
  become: true
  community.general.ufw:
    logging: true
    state: enabled
    rule: allow
    port: 22

- name: Configure UFW to allow HTTP
  become: true

```

```

community.general.ufw:
  logging: true
  state: enabled
  rule: allow
  port: 80

- name: Configure UFW to allow HTTPS
  become: true
  community.general.ufw:
    logging: true
    state: enabled
    rule: allow
    port: 443

- name: Fail2ban install
  become: true
  apt:
    state: latest
    name:
      - fail2ban

- name: Add special filters for Fail2ban
  become: true
  copy:
    src: filter.d/
    dest: /etc/fail2ban/filter.d/
    owner: root
    group: root
    mode: "0644"

- name: Configure Fail2ban jail.local
  become: true
  copy:
    src: jail.local
    dest: /etc/fail2ban/jail.local
    owner: root
    group: root
    mode: "0644"
  notify:
    - Restart Fail2ban after configuration

```

1.6 docker-debian-install.yaml

```
---
- name: Install Docker Engine
  debug:
    msg: Starting install Docker Engine...

- name: Uninstall old versions of dependencies
  become: true
  apt:
    state: absent
    name:
      - docker.io
      - docker-doc
      - docker-compose
      - podman-docker
      - containerd
      - runc

- name: Debian dependencies install
  become: true
  apt:
    state: present
    name:
      - ca-certificates
      - curl

- name: Set permissions for /etc/apt/keyrings
  become: true
  file:
    path: /etc/apt/keyrings/
    state: directory
    owner: root
    group: root
    mode: "0755"

- name: Add Docker APT key
  become: true
  get_url:
```

```
url: https://download.docker.com/linux/debian/gpg
dest: /etc/apt/keyrings/docker.asc
```

- name: Set permission for APT key
 - become: true
 - file:
 - path: /etc/apt/keyrings/docker.asc
 - owner: root
 - group: root
 - state: touch
 - mode: a+r
- name: Add Docker APT repository
 - become: true
 - apt_repository:
 - repo: deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.asc] http
 - state: present
 - update_cache: true
 - filename: docker
- name: Install Docker Engine dependencies
 - become: true
 - apt:
 - state: present
 - name:
 - docker-ce
 - docker-ce-cli
 - containerd.io
 - docker-buildx-plugin
 - docker-compose-plugin
 - notify:
 - Enable --now docker service
 - Enable --now containerd

1.7 nginx-debian-install.yaml

- name: Nginx configuration
 - debug:
 - msg: Starting Nginx Configuration...

- name: Nginx install
 - become: true
 - apt:
 - state: present
 - name:
 - nginx
 - openssl

- name: Create self-signed private key for Nginx
 - become: true
 - community.crypto.openssl_privatekey:
 - path: /etc/ssl/private/nginx-selfsigned.key
 - size: 2048
 - mode: "0600"
 - type: RSA

- name: Create self-signed cert for Nginx
 - become: true
 - community.crypto.x509_certificate:
 - path: /etc/ssl/certs/nginx-selfsigned.crt
 - privatekey_path: /etc/ssl/private/nginx-selfsigned.key
 - provider: selfsigned
 - selfsigned_not_after: +3650d
 - mode: "0644"

- name: Configure Nginx default.conf
 - become: true
 - copy:
 - src: default.conf
 - dest: /etc/nginx/conf.d/default.conf
 - owner: root
 - group: root
 - mode: "0644"
 - notify:
 - Restart Nginx after configuration

Appendix B FastAPI Back End

2.1 app/

2.1.1 configs/

a) config.py

```
from pydantic_settings import BaseSettings
import os
from dotenv import load_dotenv
# Загрузка файла среды
load_dotenv(".env")

# Настройки PostgreSQL
class PostgresSettings(BaseSettings):
    PG_USER: str = os.getenv("PG_USER")
    PG_PASSWORD: str = os.getenv("PG_PASSWORD")
    PG_SERVER: str = os.getenv("PG_SERVER")
    PG_DB: str = os.getenv("PG_DB")

# Настройки Redis
class RedisSettings(BaseSettings):
    RD_SERVER: str = os.getenv("RD_SERVER")
    RD_PASSWORD: str = os.getenv("RD_PASSWORD")
    RD_PORT: int = os.getenv("RD_PORT")
```

2.1.2 database/

a) models/user.py

```
from sqlalchemy import Column, Integer, String
from app.database.db import Base

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, nullable=False)
    name = Column(String, nullable=False)
    password = Column(String, nullable=False)
    surname = Column(String, nullable=False)
    email = Column(String, nullable=False, unique=True)
```

b) db.py

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
from sqlalchemy_utils import database_exists, create_database
from app.configs.config import PostgresSettings

postgres_settings = PostgresSettings()

# Путь к PostgreSQL
SQLALCHEMY_DATABASE_URL = f"postgresql://{postgres_settings.PG_USER}:{
                                f"{postgres_settings.PG_SERVER}/{postgres_se

# Движок
engine = create_engine(
    SQLALCHEMY_DATABASE_URL
)

# Если нет базы данных, то создать
if not database_exists(engine.url):
    create_database(engine.url)

# Открытие сессии
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=en

Base = declarative_base()

# Инициализация PostgreSQL
def init_db():
    Base.metadata.create_all(bind=engine)

# Подключение к PostgreSQL
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

c) redis.py

```
import redis.asyncio as redis
from app.configs.config import RedisSettings
```

```
redis_settings = RedisSettings()
```

```
r = redis.Redis(host=redis_settings.RD_SERVER, port=redis_settings.RD_
```

2.1.3 exceptions/

a) custom.py

```
class MyHTTPException(Exception):
    def __init__(self, code: int, success: bool, msg: str):
        self.code = code
        self.success = success
        self.msg = msg
```

2.1.4 routers/

a) v1/user_router.py

```
from fastapi import APIRouter, Depends
from fastapi.responses import JSONResponse
from sqlalchemy.orm import Session
from app.database.db import get_db
from app.database.models.user import User
from app.exceptions.custom import MyHTTPException
from app.schemas.errors.user_schema import UserCreateErrorSchema, User
from app.schemas.requests.user_schema import UserCreateSchema
from app.schemas.responses.user_schema import UserCreateResponseSchema
from app.services.user_service import UserService
```

```
import hashlib
```

```
user_router = APIRouter(tags=["Users"], prefix='/users')
```

```
@user_router.get("/get_all_users", response_model=UserGetAllResponseSc
    500: {"model": UserDBErrorSchema, "description": "Problems with DB
})
```

```
async def get_all_users(db: Session = Depends(get_db)):
    r = await UserService(db).get_all_users()
    # Если прилетел Exception
    if isinstance(r, MyHTTPException):
        return r
```

```

else:
    return JSONResponse(status_code=200, content={"success": True,

@user_router.post("/create_user", response_model=UserCreateResponseSch
    409: {"model": UserCreateErrorSchema, "description": "Conflict in
    500: {"model": UserDBErrorSchema, "description": "Problems with DB
})
async def create_user(userSchema: UserCreateSchema, db: Session = Depen
    hash_password = (hashlib.md5((userSchema.password).encode())).hexdigest
    user = User(name=userSchema.name, surname=userSchema.surname, passwor
    r = await UserService(db).create_user(user)
    # Если прилетел Exception
    if isinstance(r, MyHTTPException):
        raise r
    elif not r:
        raise MyHTTPException(code=409, success=False, msg="This email
    else:
        return JSONResponse(status_code=200, content={"success": True,

```

2.1.5 schemas/

a) errors/user_schema.py

```
from pydantic import BaseModel
```

```

class UserDBErrorSchema(BaseModel):
    success: bool = False
    msg: str = 'Problems with DB...'

```

```

class UserCreateErrorSchema(BaseModel):
    success: bool = False
    msg: str = 'This email was registered before'

```

b) requests/user_schema.py

```
from pydantic import BaseModel, EmailStr
```

```

class UserCreateSchema(BaseModel):
    name: str = 'Abay'
    surname: str = 'Sadykov'

```

```
password: str = "12345678"
email: EmailStr = 'pythondelay@gmail.com'
```

2.1.6 responses/user_schema.py

```
from typing import List
from pydantic import BaseModel

class UserCreateResponseSchema(BaseModel):
    success: bool = True
    msg: str = 'User was registered'
```

```
class UserGetAllResponseSchema(BaseModel):
    success: bool
    msg: List[dict]
```

2.1.7 services/

a) sql_service.py

```
from fastapi import Depends
from sqlalchemy.orm import Session
from app.database.db import get_db
```

```
# SQL-сервисы
```

```
class SQLService:
    def __init__(self, db: Session = Depends(get_db)):
        self.db = db
```

b) user_service.py

```
from sqlalchemy import exc
from app.database.models.user import User
from app.exceptions.custom import MyHTTPException
from app.services.sql_service import SQLService
from app.utils import sqlalchemy_object_to_dict
```

```
class UserService(SQLService):
    async def create_user(self, user: User):
        r = await self.check_user_by_email(user)
        if not r:
            try:
```

```

        self.db.add(user)
        self.db.commit()
        return True
    except exc.SQLAlchemyError as e:
        print(e)
        raise MyHTTPException(code=500, success=False, msg="Pr
else:
    return False

async def check_user_by_email(self, user: User):
    try:
        q = self.db.query(User).filter(User.email == user.email).f
        if q:
            return True
        else:
            return False
    except exc.SQLAlchemyError as e:
        print(e)
        raise MyHTTPException(code=500, success=False, msg="Proble

async def get_user_by_email(self, user: User):
    try:
        q = self.db.query(User).filter(User.email == user.email).f
        if q:
            user = await sqlalchemy_object_to_dict(q)
            return user
        else:
            return {}
    except exc.SQLAlchemyError as e:
        print(e)
        raise MyHTTPException(code=500, success=False, msg="Proble

async def get_all_users(self):
    try:
        # Отправить запрос
        q = self.db.query(User).all()
        list_of_users = []
        # Преобразовать его в список
        for i in q:
            user = await sqlalchemy_object_to_dict(i)

```

```

        list_of_users.append(user)
    return list_of_users
except exc.SQLAlchemyError as e:
    print(e)
    raise MyHTTPException(code=500, success=False, msg="Problem")

```

2.1.8 main.py

```

from fastapi import FastAPI, APIRouter, Request
from fastapi.responses import JSONResponse
from fastapi.middleware.cors import CORSMiddleware

from app.database.db import init_db
from app.routers.v1.user_router import user_router
from app.exceptions.custom import MyHTTPException

# Создание приложения на пути /api/
app = FastAPI(docs_url="/api/docs", redoc_url="/api/redoc", openapi_url="/api/openapi.json")

# Подключение CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

# Общий роутер, на который цепляем все остальные endpoint-ы
api_router = APIRouter(prefix="/api")

# Корневой endpoint
@api_router.get("/")
async def root():
    return {"message": "FastAPI template of LaggerIsME(https://github.com/LaggerIsME)"}

# Действия при старте приложения
@app.on_event("startup")
async def startup():
    init_db()

```

```

# Кастомные исключения
@app.exception_handler(MyHTTPException)
async def unicorn_exception_handler(request: Request, exc: MyHTTPException):
    return JSONResponse(status_code=exc.code, content={"success": exc.

# Добавляем роутеры на общий /api
api_router.include_router(user_router)
# Добавляем общий роутер к приложению
app.include_router(api_router)

```

2.1.9 utils.py

```

# Преобразовать SQLAlchemy-объект в Python dict
async def sqlalchemy_object_to_dict(sqlalchemy_object):
    sqlalchemy_object_dict = sqlalchemy_object.__dict__
    del sqlalchemy_object_dict['_sa_instance_state']
    return sqlalchemy_object_dict

```

2.2 .env

```

# FastAPI settings
APP_PORT="8000"
# PostgreSQL settings
PG_USER="postgres"
PG_PASSWORD="12345678"
PG_SERVER="postgresql"
PG_DB="db_test"
PG_PORT="5432"
# Redis settings
RD_SERVER="redis"
RD_PASSWORD="12345678"
RD_PORT="6379"

```

2.3 requirements.txt

```

pydantic==2.4.2
pydantic-settings==2.0.3
python-dotenv==1.0.0
fastapi==0.104.0
SQLAlchemy==2.0.22
redis==5.0.1
psycopg2-binary==2.9.9

```

```
SQLAlchemy-Utills==0.41.1
uvicorn==0.23.2
email-validator==2.1.0.post1
redis==5.0.1
```

2.4 Dockerfile

```
# Выбор менее прожорливого образа
FROM python:3.10-slim-bullseye
# Установка CURL
RUN apt-get -y update
RUN apt-get -y install curl
# Создание рабочей директории
WORKDIR /code
# Копирование зависимостей
COPY ./requirements.txt /code/requirements.txt
# Обновление инструментов
RUN pip install --no-cache-dir --upgrade setuptools
# Установка зависимостей
RUN pip install -r /code/requirements.txt
# Копирование файлов
COPY . .
```

2.5 docker-compose.yml

```
version: "3.9"
```

```
services:
```

```
  fastapi:
```

```
    build: .
```

```
    container_name: fastapi
```

```
    command: bash -c "uvicorn app.main:app --host 0.0.0.0 --port ${APP_PORT}"
```

```
    env_file:
```

```
      - .env
```

```
    ports:
```

```
      - 127.0.0.1:${APP_PORT}:${APP_PORT}
```

```
    depends_on:
```

```
      # Проверка состояния PostgreSQL
```

```
      postgresql:
```

```
        condition: service_healthy
```

```
      # Проверка состояния Redis
```

```
      redis:
```

```
        condition: service_healthy
```

```

healthcheck:
  test: [ "CMD-SHELL", "curl --fail http://localhost:${APP_PORT}/a
  interval: 40s
  timeout: 30s
  retries: 3
  start_period: 60s

restart: on-failure

postgresql:
  image: postgres:15
  container_name: postgresql
  env_file:
    - .env
  environment:
    POSTGRES_PASSWORD: ${PG_PASSWORD}
    POSTGRES_USER: ${PG_USER}
  ports:
    - 127.0.0.1:${PG_PORT}:${PG_PORT}

healthcheck:
  test: [ "CMD-SHELL", "pg_isready -p ${PG_PORT} -U ${PG_USER}" ]
  interval: 1s
  timeout: 5s
  retries: 5

restart: on-failure

redis:
  image: redis:7
  command: redis-server --requirepass ${RD_PASSWORD}
  container_name: redis
  env_file:
    - .env
  ports:
    - 127.0.0.1:${RD_PORT}:${RD_PORT}

healthcheck:
  test: [ "CMD-SHELL", "redis-cli ping -a ${RD_PASSWORD}" ]

```

```
interval: 1s  
timeout: 5s  
retries: 5
```

```
restart: on-failure
```

Appendix C Gitlab CI/CD

3.1 .gitlab-ci.yml

stages:

- secret_detection
- build
- sast
- sca

include:

- template: Jobs/Secret-Detection.gitlab-ci.yml
- template: Security/SAST.gitlab-ci.yml

build_project:

stage: build

Start job only manual

when: manual

script:

Change branch to right local repository

- git checkout master

Update local repository

- git pull

Rebuild docker containers

- docker compose up -d --build

Create artifact

- tar -czvf project.tar.gz *

artifacts:

when: on_success

access: all

expire_in: 30 days

paths:

- ./project.tar.gz

tags:

- native

secret_detection:

stage: secret_detection

variables:

SECRET_DETECTION_HISTORIC_SCAN: "true"

Number of leaks

```

    GET_VULNERABILITY_COUNT: "cat gl-secret-detection-report.json | jq
# Do not allow failure
tags:
  - docker
script:
  # Install JQ for parsing data
  - apk add jq
  # Start gitleaks
  - /analyzer run
  # show number
  - exit $(eval "$GET_VULNERABILITY_COUNT")

sast:
  stage: sast

semgrep-sast:
  stage: sast
  artifacts:
    reports:
      sast: gl-sast-report.json
  needs:
    - build_project
  tags:
    - docker

scancode-toolkit_check:
  stage: sca
  tags:
    - native
  script:
    - /opt/scancode-toolkit-v32.1.0/scancode ./ --json-pp ./dependency
  artifacts:
    when: always
    access: all
    expire_in: 30 days
    paths:
      - ./dependency-check-report.json
  needs:
    - semgrep-sast

```